

Lovedis.de mit Mara-Plattform

Architektur- & Sicherheitskonzept für die Lovedis-Plattform und die Mara-Anwendungsschicht

VERTRAULICH · SCHUTZKLASSE K3

STAND

31. Juli 2026

VERSION

1.0 — Planungsdokument

GELTUNGSBEREICH

lovedis.de · Mara-Plattform · Data Warehouse

NÄCHSTE PRÜFUNG

Januar 2027

Sicherheits- & Data-Warehousing-Konzept — Lovedis.de mit Mara-Plattform

Dokumenttyp: Architektur- & Sicherheitskonzept (Planungsdokument) **Stand:** 31.07.2026 · **Repo:**

`/Users/tim Eggert/Documents/Lovedis` · **Branch-Basis:** `mara` **Geltungsbereich:** Öffentliche Homepage `Love-dis.de`, die Mara-Plattform (Screening, Partner-SSOT, Marktplatz, Venture-Credits, Match-Matrix), die zugehörige PostgreSQL-Datenhaltung, das Data Warehouse sowie alle betrieblichen Prozesse (Hosting, Secrets, Passwörter, Backup, Monitoring). **Nicht im Geltungsbereich:** Marketing-Tooling ohne Personenbezug, Endgeräte-Management (nur gestreift in §10.6).

Inhalt

- | | |
|---|--|
| 1. Management-Summary & Empfehlung | 12. Anwendungssicherheit |
| 2. Ist-Analyse und konkrete Findings | 13. Edge-, Netz- & Transportsicherheit |
| 3. Schutzbedarfsfeststellung & Datenklassifizierung | 14. Mara-Integration: Datenflüsse & Fremdsysteme |
| 4. Bedrohungsmodell (STRIDE) | 15. Data Warehousing |
| 5. Hosting: offener Optionsvergleich & Empfehlung | 16. Logging, Monitoring & Incident Response |
| 6. Ziel-Architektur im Detail | 17. DSGVO & Compliance |
| 7. Datenbank-Sicherheit & -Betrieb | 18. CI/CD & Supply-Chain-Sicherheit |
| 8. Backup, PITR & Disaster Recovery | 19. Umsetzungs-Roadmap |
| 9. Identität, Authentifizierung & Sessions | 20. Offene Entscheidungen |
| 10. Passwort- & Secret-Management | 21. Anhang |
| 11. Autorisierung, RBAC-Härtung & Audit | |

1. Management-Summary & Empfehlung

1.1 Worum es geht

Lovedis betreibt zwei Dinge unter einer Marke: die **öffentliche Homepage** `lovedis.de` und die **Mara-Plattform** — eine Multi-Rollen-Anwendung, in der interne Scouts, Partnerunternehmen, Investoren und Startups zusammenarbeiten. Die Plattform hält **wirtschaftlich hochsensible Daten**:

- **Interne Bewertungen** von Startups (7-dimensionale Scorings, Empfehlungen `STRONG_YES` ... `STRONG_NO`).
- Die **Match-Matrix** — eine cross-partner Bewertung, welches Startup zu welchem Partnerunternehmen passt, inklusive Kontaktstatus und Next Steps. **Das ist der kritischste Datensatz im System:** Ein Leak legt gegenüber jedem Partner offen, wie er relativ zu seinen Wettbewerbern eingeordnet wird und mit wem er in Gesprächen steht.
- **Partner-Verdikte** (`PartnerStartupReview`) — Partner A darf nie sehen, wie Partner B votet.
- **Startup-Internia:** Funding, Pitchdecks, Kontaktdaten, Pipeline-Stände.
- **1:1-Nachrichten** (`Message`) im Klartext.
- Ein **Credit-Ledger** mit Geldwert-Charakter (`CreditAccount` / `CreditTransaction`).

Der Schutzbedarf ist damit **hoch** — nicht wegen Massen-PII, sondern wegen **Vertraulichkeit von Geschäftsgeheimnissen Dritter**. Genau danach werden mittelständische Partner (FingerHaus, Lupp, Weimer, INNEXIS, Sälzer) im Einkaufs- bzw. Compliance-Prozess fragen.

1.2 Kernaussage zur Hosting-Frage

Der aktuelle Stack — **Next.js auf Cloudflare Workers (OpenNext) + Neon Serverless Postgres** — ist für einen *Staging-Durchstich* eine gute Wahl: schnell, günstig, kein Server-Betrieb. Für den **produktiven Betrieb einer B2B-Plattform mit deutschen Industriepartnern ist er nicht optimal**. Die drei ausschlaggebenden Gründe:

1. **Datensouveränität.** Cloudflare Inc. und Neon (seit der Databricks-Übernahme) sind US-Unternehmen und damit dem **CLOUD Act** unterworfen. Eine EU-Region reicht dafür nicht; maßgeblich ist die Beherrschbarkeit durch den Konzern. Cloudflare-Traffic läuft im Standard-Setup über das nächstgelegene PoP **weltweit**; eine echte EU-Bindung erfordert die *Data Localization Suite* (kostenpflichtiges Enterprise-Add-on). Für jeden Partner-Compliance-Fragebogen ist das eine erklärungsbedürftige Position, für eine öffentlich geförderte oder kommunal angebundene Hub-Struktur ggf. ein Ausschlusskriterium.
2. **Die Workers-Runtime erzwingt schwächere Krypto.** `bcryptjs` (reines JavaScript) ist im Repo im Einsatz, **weil** die Workers-Runtime keine nativen Node-Module lädt. `bcryptjs` ist rund eine Größenordnung langsamer als eine native Implementierung — was in der Praxis bedeutet, dass man den Cost-Faktor niedrig halten *muss*, um die CPU-Zeitgrenze des Workers nicht zu reißen. Damit ist **Argon2id — der Stand der Technik für Passwort-Hashing — auf Workers praktisch nicht umsetzbar**. Auf einer Node-Runtime ist es ein Einzeiler (`@node-rs/argon2`). Die Hosting-Wahl diktiert hier direkt das Sicherheitsniveau der Authentifizierung.
3. **Betriebs- und Forensikfähigkeit.** Workers bieten keine persistenten Prozesse, kein Dateisystem, stark limitierte CPU-Budgets und nur eingeschränkte Log-Retention. Für Audit-Trails, langlaufende ETL-Jobs, PDF-/Excel-Exporte serverseitig, forensische Analysen und Incident Response ist das ein struktureller Nachteil. Zusätzlich zwingt das Bundle-Limit bereits heute zu Workarounds (`compilerBuild = "small"` im Prisma-Schema, `outputFileTracingIncludes` für `pg-cloudflare`).

1.3 Empfehlung

Ziel-Hosting: Szenario A — „EU-souverän“ (§5.3). Next.js im **Node-Standalone-Container** auf **Hetzner Cloud (Frankfurt/Nürnberg, DE)**, zwei App-Nodes hinter einem Hetzner Load Balancer, **managed PostgreSQL 17 in der EU** (Empfehlung: Aiven Frankfurt oder IONOS/STACKIT für rein deutsche Vertragskette) mit HA-Standby und PITR, **privates Netz** zwischen App und DB, Objektspeicher für Uploads in DE, **Myra Security (BSI-C5/§8a-zertifiziert, DE)** als WAF/DDoS-Schicht vor der Anwendung. Getrenntes **DWH-Postgres** mit dbt und selbst gehostetem Metabase.

Damit erreicht man gleichzeitig: vollständige EU-/DE-Vertragskette, Argon2id, echte Audit-Trails, freie Wahl der ETL-Werkzeuge, planbare Kosten (~140–260 €/Monat für Produktion inkl. HA-DB) und einen Compliance-Fragebogen, der sich in einem Satz beantworten lässt.

Cloudflare bleibt optional als reiner DNS-Provider (kein Proxy) oder wird komplett ersetzt. Wenn man Cloudflare als WAF behalten will, ist das vertretbar — dann aber bewusst und mit dokumentiertem Transfer-Impact-Assessment (§17.5).

Wenn kein Migrationsbudget besteht, ist Szenario B (§5.4) die Rückfallposition: Cloudflare Workers + Neon, aber mit Region-Pinning Frankfurt, Data Localization Suite und der bewussten Inkaufnahme von bcrypt statt Argon2id. Das ist *betreibbar*, aber nicht *optimal* — und diese Unterscheidung sollte dokumentiert und vom Management abgenommen werden.

1.4 Top-Risiken (vor Umsetzung dieses Konzepts)

#	Risiko	Eintritt	Schaden	Brutto	\$
R1	Seed-Passwort <code>Lovedis2026!</code> gelangt in Produktion; Demo-Accounts (<code>admin@lovedis.dev</code>) bleiben aktiv	Mittel	Kritisch	Sehr hoch	\$2.1
R2	Kein Rate-Limiting / kein Lockout am Credentials-Login → Credential Stuffing gegen Admin-Konten	Hoch	Kritisch	Sehr hoch	\$9.4
R3	Match-Matrix / Partner-Verdikte durch Rollen-Fehler oder IDOR sichtbar für falschen Partner	Mittel	Kritisch	Sehr hoch	\$11
R4	Kein Audit-Log — Datenabfluss ist nachträglich nicht nachweisbar (auch DSGVO Art. 5 Abs. 2)	Hoch	Hoch	Hoch	\$11.5
R5	JWT-only Sessions: kein serverseitiger Widerruf, kompromittiertes Token gilt bis Ablauf	Mittel	Hoch	Hoch	\$9.5
R6	Anhänge/Media nur als URL — keine Zugriffskontrolle auf Dateiebene	Hoch	Hoch	Hoch	\$12.6
R7	<code>xlxsx@0.18.5</code> mit bekannten Prototype-Pollution-/ReDoS-Schwachstellen	Hoch	Mittel	Hoch	\$18.4
R8	<code>next-auth@5.0.0-beta.31</code> — Beta-Abhängigkeit im Auth-Pfad in Produktion	Mittel	Hoch	Hoch	\$18.3
R9	Keine Security-Header / keine CSP → XSS-Auswirkung unbegrenzt	Mittel	Mittel	Mittel	\$12.2
R10	Kein dokumentiertes Restore, kein Restore-Test → Backup-Wert unbewiesen	Mittel	Kritisch	Hoch	\$8

2. Ist-Analyse und konkrete Findings

2.1 Ist-Architektur (verifiziert am Code)

Schicht	Umsetzung	Datei/Fundstelle
Framework	Next.js 16 App Router, React 19, RSC + Server Actions, Turbopack	<code>package.json</code>
Auth	NextAuth v5 Beta, Credentials-Provider, JWT-Sessions , <code>bcryptjs</code>	<code>src/auth.ts</code> , <code>src/auth.config.ts</code>
Edge-Gate	<code>middleware.ts</code> — reiner JWT-Check, Public-Path-Allowlist	<code>src/middleware.ts</code>
Autorisierung	<code>requireAuth</code> / <code>requireRole</code> / <code>requireScoutModule</code> / <code>requirePartner</code> ...	<code>src/lib/auth-guards.ts</code>
Schreib-API	ausschließlich Server Actions (<code>src/app/actions/*</code>); REST nur <code>/api/auth</code> , <code>/api/health</code> , <code>/api/cron</code> , <code>/api/session-clear</code>	README, <code>src/app/actions/</code>
ORM/DB	Prisma 7, PostgreSQL 17, <code>@prisma/adapters-pg</code> lokal / <code>@prisma/adapters-neon</code> im Worker	<code>prisma/schema.prisma</code>
Hosting	OpenNext → Cloudflare Worker <code>lovedis-platform-staging</code> , <code>workers_dev: true</code> , Account <code>e29e8d60...</code> (getrennt vom Homepage-Account <code>9337dd36...</code>)	<code>wrangler.jsonc</code>
Secrets	<code>wrangler secret put</code> , <code>.env</code> gitignored	<code>docs/deployment-plan-mara.md</code>
Rollen	<code>ADMIN</code> , <code>MEMBER</code> , <code>BUSINESS_PARTNER</code> , <code>INVESTOR</code> , <code>STARTUP</code>	<code>schema.prisma</code>

Positiv hervorzuheben (das ist bereits solide gebaut und bleibt so):

- `requireAuth()` liest die Rolle **frisch aus der DB** und überschreibt den JWT-Snapshot. Eine Degradierung (Admin → Member) wirkt damit ab dem nächsten Request. Das ist genau richtig gelöst.
- Es wird geprüft, ob der User aus dem JWT noch existiert und `isActive` ist; ungültige Sessions gehen über `/api/session-clear` , um Redirect-Loops zu vermeiden.
- Der Partner-Freigabe-Gate (`approvedAt` null → `/pending`) existiert und wird zusätzlich defense-in-depth in Write-Actions geprüft (`isPartnerApproved`).
- Der Cron-Endpoint **fail-closed in Produktion**: ohne `CRON_SECRET` gibt es in `production` keinen Bypass, und `GET` ist bewusst nebenwirkungsfrei (405).
- Server Actions als einzige Schreib-API mit Zod-Validierung und erneuter `auth()` -Prüfung pro Action ist ein sehr gutes Muster — es gibt kaum ungeschützte Angriffsfläche durch vergessene REST-Routen.
- Der Credit-Ledger arbeitet mit `prisma.$transaction` , conditional `updateMany` -Guards und Doppelbuchungsschutz über `creditTransactionId` . Finanzlogik ist damit race-safe.
- Die Staging-Isolation (eigener CF-Account, keine Routes, eigene DB) ist saubere Praxis.

2.2 Findings mit Schweregrad

Bewertung: **S1** = vor Go-Live zwingend, **S2** = zeitnah, **S3** = Verbesserung.

ID	Sev	Finding	Belegstelle	Maßnahme
F-01	S1	Einheitliches Seed-Passwort <code>Lovedis2026!</code> für alle Demo-Rollen ist im README dokumentiert. Wird der Seed je gegen Produktion gefahren, existieren bekannte Admin-Zugänge.	<code>README.md</code> , <code>prisma/seed.ts</code>	Seed in Prod hart blocken (§7.7); Prod-Admin nur via Einmal-Invite
F-02	S1	Kein Rate-Limiting, kein Account-Lockout, kein CAPTCHA am Login. <code>authorize()</code> ist unbegrenzt aufrufbar.	<code>src/auth.ts:21</code>	§9.4
F-03	S1	Keine MFA für <code>ADMIN</code> / <code>MEMBER</code> , obwohl diese Rollen die Match-Matrix und alle Bewertungen sehen.	—	§9.3
F-04	S1	Kein Passwort-Reset-Flow. Folge in der Praxis: Admins setzen Passwörter manuell und versenden sie über unsichere Kanäle.	keine Fundstelle	§9.6
F-05	S1	Keine Security-Header, keine CSP. <code>next.config.ts</code> enthält keinen <code>headers()</code> -Block.	<code>next.config.ts</code>	§12.2
F-06	S1	Kein Audit-Log. Kein Modell im Schema erfasst, wer wann welche Matrix-Zelle, welches Verdikt oder welche Credit-Buchung gelesen/geändert hat.	<code>schema.prisma</code>	§11.5
F-07	S1	<code>xlsx@0.18.5</code> (SheetJS via npm) — Prototype Pollution (CVE-2023-30533) und ReDoS (CVE-2024-22363). Wird für Excel-Import/-Export benutzt, d. h. verarbeitet fremde Dateien .	<code>package.json:46</code>	§18.4
F-08	S2	JWT-only Sessions ohne serverseitige Widerrufsmöglichkeit; kein <code>Session</code> -Modell, keine Token-Versionierung.	<code>auth.config.ts:12</code>	§9.5
F-09	S2	<code>bcryptjs</code> statt Argon2id; Cost-Faktor durch Workers-CPU-Limit nach oben begrenzt.	<code>src/auth.ts:3</code>	§9.2
F-10	S2	<code>Attachment.url</code> / <code>MediaAsset.url</code> sind freie URLs ohne Zugriffskontrolle und ohne Validierung des Schemas (SSRF-/Phishing-Vektor über <code>javascript:</code> - oder interne URLs).	<code>schema.prisma:425</code>	§12.6
F-11	S2	<code>Message.body</code> , <code>Contact.email</code> , <code>Contact.phone</code> unverschlüsselt in der DB (nur Transport- und Storage-Encryption).	<code>schema.prisma:583</code> , <code>:410</code>	§7.5
F-12	S2	Keine Row-Level Security in Postgres. Autorisierung existiert nur in der Applikationsschicht; ein einziger vergessener <code>requireRole</code> legt Daten offen.	—	§7.4
F-13	S2	<code>next-auth@5.0.0-beta.31</code> — Beta im sicherheitskritischen Pfad.	<code>package.json:40</code>	§18.3
F-14	S2	Keine Passwort-Policy und kein Breach-Check bei Selbstregistrierung (<code>/auth/signup/partner</code> , <code>/auth/signup/startup</code>).	<code>src/app/actions/auth.ts</code>	§9.7
F-15	S2	Kein dokumentiertes/getestetes Restore-Verfahren; RPO/RTO nicht definiert.	—	§8
F-16	S3	<code>auth.ts</code> lädt bei Login die komplette User-Row (kein <code>select</code>), inkl. aller Felder. Unnötige Datenexposition im Speicher/Log.	<code>src/auth.ts:25</code>	<code>select</code> einschränken
F-17	S3	Kein Historien-/Event-Modell für <code>Startup.pipelineStage</code> — Stage-Wechsel sind nicht rekonstruierbar. Blockiert Funnel-Analytics (§15.4) und Audit.	<code>schema.prisma:344</code>	§15.4

ID	Sev	Finding	Belegstelle	Maßnahme
F-18	S3	<code>/odie</code> ist als Public Path freigeschaltet („Easter Egg“), erreichbar mit und ohne Session.	<code>src/middleware.ts:14</code>	Prüfen, ob in Prod nötig
F-19	S3	Keine Begrenzung der Session-Lebensdauer konfiguriert (NextAuth-Default 30 Tage).	<code>auth.config.ts:12</code>	§9.5
F-20	S3	<code>html2canvas</code> / <code>jspdf</code> / <code>xlsx</code> erzeugen Exporte clientseitig — Exporte sind damit nicht auditierbar und nicht wasserzeichenfähig.	<code>package.json</code>	§11.6

3. Schutzbedarfsfeststellung & Datenklassifizierung

3.1 Klassen

Klasse	Bezeichnung	Definition	Anforderung
K0	Öffentlich	Bewusst veröffentlicht (Homepage, <code>Startup.isPublished</code> -Storefront)	Integrität
K1	Intern	Betrieblich, ohne Dritt-Geheimnis (Roadmap, ContentPage, Knowledge-Board)	Zugriff nur mit Login
K2	Vertraulich	Personenbezug oder Geschäftsgeheimnis einer Partei	Rollen-Gate + Audit
K3	Streng vertraulich	Cross-Partner-Geheimnisse, Bewertungen, Credentials	Rollen-Gate + Audit + Verschlüsselung + MFA-Pflicht der Zugreifenden

3.2 Zuordnung je Modell

Modell / Feld	Klasse	Begründung	Aufbewahrung
<code>User.passwordHash</code>	K3	Credential	bis Kontolöschung
<code>User.email</code> , <code>.name</code> , <code>.company</code>	K2	PII	Konto + 30 Tage
<code>PartnerStartupMatch.*</code>	K3	Cross-Partner-Wettbewerbsinformation	3 Jahre
<code>PartnerStartupReview.verdict/.note</code>	K3	Votum eines Partners über ein Startup	3 Jahre
<code>Evaluation</code> , <code>Score</code>	K3	Interne Bewertung, geschäftskritisch	5 Jahre
<code>Startup.screenSummary/.screenRecommendation</code>	K3	Interne Erst-Einordnung	5 Jahre
<code>Startup.fundingRaised</code> , <code>.seekingAmount</code>	K2	Finanzdaten Dritter	5 Jahre
<code>Contact.email/.phone/.notes</code>	K2	PII Dritter	bis Widerruf
<code>Message.body</code>	K3	Vertrauliche 1:1-Kommunikation	12 Monate, dann Löschung
<code>CreditAccount</code> , <code>CreditTransaction</code>	K2	Geldwert, buchhalterisch relevant	10 Jahre (§ 147 AO)
<code>MarketplaceBooking.contactEmail/.message</code>	K2	PII + Anliegen	3 Jahre
<code>Attachment</code> , <code>MediaAsset</code>	K2–K3	Pitchdecks, interne Dokumente	wie Startup
<code>IntroRequest.message</code>	K2	Investor-Interesse	2 Jahre
<code>StartupPush</code> , <code>CheckInReminder</code>	K2	interner Funnel	2 Jahre
<code>RoadmapItem</code> , <code>ContentPage</code> , <code>KnowledgeResource</code>	K1	Redaktionell	unbegrenzt
Audit-Log (neu, §11.5)	K2	Nachweispflicht	12 Monate

Konsequenz: Es gibt **K3-Daten in nahezu jedem Kern-Workflow**. Das Konzept muss deshalb durchgehend von „streng vertraulich“ ausgehen, nicht von „normalem SaaS“.

3.3 Schutzbedarf nach BSI-Schema

Ziel	Bedarf	Begründung
Vertraulichkeit	Hoch	Cross-Partner-Matrix, Bewertungen, Nachrichten. Ein Leak beschädigt die Geschäftsbeziehung zu allen Partnern gleichzeitig und ist irreversibel.
Integrität	Hoch	Credit-Ledger hat Geldwert; manipulierte Scores steuern Investitionsentscheidungen.
Verfügbarkeit	Normal	Kein Echtzeit-Betriebsprozess. Ausfall < 8 h ist verkraftbar (→ RTO 4 h, §8.2).

4. Bedrohungsmodell (STRIDE)

4.1 Angreifer-Profile

Profil	Motivation	Fähigkeit	Relevanz
A1 — Opportunistischer Bot	Credential Stuffing, Ressourcen-Missbrauch	Automatisiert, breit	Hoch (F-02)
A2 — Neugieriger Partner-User	Wissen, wie Wettbewerber bewertet werden	Legitimes Login, manipuliert IDs/URLs	Sehr hoch
A3 — Startup-User	Eigene Bewertung/Ranking sehen	Legitimes Login	Hoch
A4 — Ex-Mitarbeiter	Datenmitnahme (Exporte)	Ehemals Admin, kennt Struktur	Hoch
A5 — Gezielter Angreifer	Deal-Informationen, Erpressung	Phishing, Supply Chain	Mittel
A6 — Insider-Fehler	keine (Versehen)	Vollzugriff	Hoch
A7 — Provider/Behörde	Rechtsanordnung (CLOUD Act)	Infrastrukturzugriff	Mittel — treibt \$5

4.2 STRIDE-Matrix

Kategorie	Konkrete Bedrohung in Lovedis/Mara	Aktuell	Zielmaßnahme
Spoofing	Credential Stuffing gegen <code>admin@lovedis.dev</code> ; Session-Cookie-Diebstahl per XSS	bcrypt-Vergleich, sonst nichts	Argon2id, MFA, Rate-Limit, <code>__Host-</code> -Cookie, CSP (§9, §12.2)
Tampering	Manipulation von <code>Score.value</code> , <code>CreditTransaction.amount</code> , Matrix-Zellen; Mass Assignment in Server Actions	Zod pro Action, <code>\$transaction</code> - Guards	Explizite Allowlists, Audit-Log, DB-Constraints (§7.3, §11.5)
Repudiation	„Ich habe dieses Verdikt nie abgegeben“; unklarer Datenabfluss	<code>updatedById</code> nur als letzter Stand	Append-only Audit-Log (§11.5)
Information Disclosure	A2/A3 liest fremde Matrix-Zeilen/Verdikte ; IDOR auf <code>Attachment.url</code> ; Exporte; DB-Dump	Rollen-Gates in der App	RLS (§7.4), signierte URLs (§12.6), Feldverschlüsselung (§7.5), Export-Audit (§11.6)
Denial of Service	Login-Flooding, teure Report-/Matrix-Queries, unbegrenzte Excel-Uploads	keine Limits	WAF-Rate-Limits, Query-Timeouts, Upload-Limits (§13.3, §7.6)
Elevation of Privilege	Vergessener <code>requireRole</code> in neuer Action; Rollen-Wechsel im JWT	<code>requireAuth</code> liest Rolle frisch (gut)	RLS als 2. Schicht, Guard-Lint-Regel, Review-Pflicht (§11.3)

4.3 Wichtigster Angriffspfad (Prioritätsbegründung)

A1 Bot → /login ohne Rate-Limit (F-02)
 → Admin-Konto mit Seed-Passwort (F-01)
 → ADMIN-Rolle
 → /matrix (Match-Matrix, K3) + Excel-Export (clientseitig, kein Audit F-20)
 → vollständiger Cross-Partner-Datenabfluss, forensisch nicht nachweisbar (F-06)

Dieser Pfad besteht aus vier Findings, die **einzel**n je als „nur“ **Konfigurationsdetail** wirken. Deshalb sind F-01, F-02, F-03 und F-06 in §19 als P0 gebündelt.

5. Hosting: offener Optionsvergleich & Empfehlung

5.1 Bewertungskriterien und Gewichtung

#	Kriterium	Gewicht	Warum für Lovedis relevant
K1	Datensouveränität EU/DE (Vertragskette, CLOUD-Act-Exposition, Datenresidenz)	25 %	Deutsche Industriepartner, K3-Daten, Compliance-Fragebögen
K2	Sicherheitsfähigkeit (Krypto-Freiheit, Netzsegmentierung, Audit/Forensik, Härbarkeit)	20 %	Argon2id, Audit-Log, IR-Fähigkeit
K3	Betriebsaufwand (Ops-Stunden/Monat, benötigtes Know-how)	15 %	Kleines Team
K4	Kosten (TCO 12 Monate inkl. Ops-Zeit)	10 %	Startup-Budget
K5	Passung zum Stack (Next.js 16, Prisma 7, Node-Abhängigkeiten)	15 %	Migrationsrisiko
K6	DWH-/Analytics-Tauglichkeit (Replikation, langlaufende Jobs, BI-Anbindung)	10 %	§15
K7	Verfügbarkeit/HA & Exit (Lock-in, Portabilität)	5 %	Strategische Beweglichkeit

5.2 Übersicht der bewerteten Optionen

Bewertet wurden sieben realistische Varianten:

Variante	Kurzbeschreibung	Ergebnis
A	Hetzner DE (Container) + managed EU-Postgres + DE-WAF	Empfehlung
B	Cloudflare Workers + Neon (Status quo, gehärtet)	Rückfallposition
C	AWS eu-central-1 (ECS Fargate + Aurora Postgres)	Alternative bei Enterprise-Anforderung
D	Vercel + Neon EU	verworfen
E	STACKIT / Open Telekom Cloud (voll-deutsche Cloud)	verworfen (DX/Reifegrad)
F	Scaleway / OVHcloud (FR)	gute Alternative zu A
G	Eigenes Blech / Colocation	verworfen (Aufwand)

Verworfenen Varianten kurz begründet:

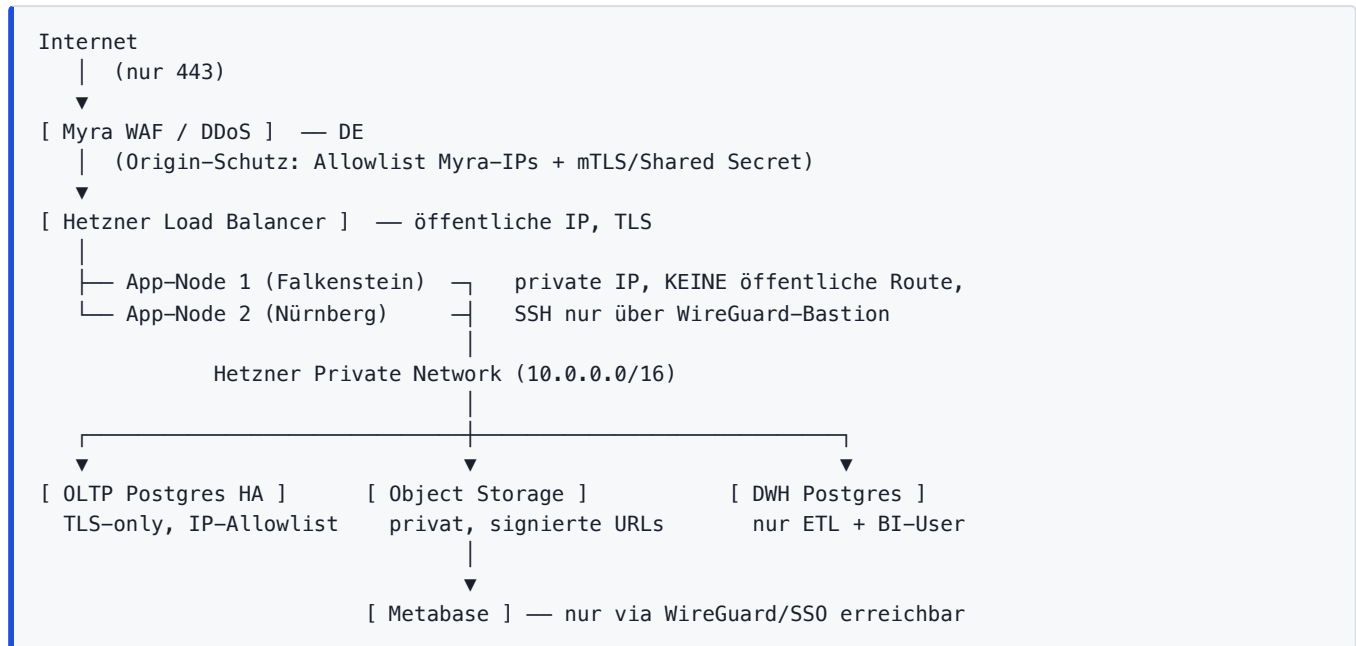
- **D (Vercel + Neon EU)** — beste Developer Experience, aber: US-Vertragskette wie B *ohne* deren Kostenvorteil, Funktions-Timeouts limitieren ETL, teure Skalierung, zusätzlicher Auftragsverarbeiter ohne Sicherheitsgewinn. Kein Kriterium, bei dem Vercel A oder B schlägt.
- **E (STACKIT/OTC)** — maximal saubere deutsche Vertragskette (Schwarz Gruppe bzw. T-Systems), aber Managed-Postgres- und Container-Angebote sind weniger reif, Tooling/Terraform-Ökosystem dünner, Support langsamer. **Wenn** ein Partner „ausschließlich deutsche Cloud“ vertraglich fordert, ist E der Upgrade-Pfad von A — technisch identische Architektur, nur anderer Anbieter.
- **G (Colocation/Eigenblech)** — beste Souveränität, aber Hardware-Lifecycle, Ersatzteile, physische Sicherheit und 24/7-Bereitschaft sind für ein Team dieser Größe nicht leistbar. Das Restrisiko (einzelner Ausfall ohne Reaktionsfähigkeit) ist höher als der Souveränitätsgewinn.

5.3 Szenario A — „EU-souverän" (EMPFEHLUNG)

Aufbau

Komponente	Wahl	Standort	Begründung
Anwendung	Next.js 16 <code>output: "standalone"</code> im Docker-Container, 2 × CPX31 (4 vCPU/8 GB)	Hetzner Falkenstein + Nürnberg	Node-Runtime → Argon2id, keine Bundle-Limits, volle Observability
Orchestrierung	Docker Compose + Kamal (oder k3s, wenn später mehr Services)	—	Bewusst <i>kein</i> Kubernetes zu Beginn: geringere Angriffsfläche und weniger Ops-Last
Reverse Proxy	Hetzner Load Balancer → Caddy je Node (auto-TLS, HTTP/3)	DE	TLS-Terminierung in Deutschland , nicht bei einem US-CDN
Datenbank (OLTP)	Managed PostgreSQL 17 , HA-Primary + Standby, PITR 14 Tage	Aiven Frankfurt (EU-Konzern, ISO 27001/SOC 2) — Alternative: IONOS DBaaS / STACKIT für 100 % DE	Managed ≠ Ops-Last für PITR/Failover; EU-Konzernsitz
Datenbank (DWH)	Separate Postgres-17-Instanz (kleiner), logische Replikation vom OLTP	gleiche Region	Trennung Analytics/Produktion (§15)
Objektspeicher	Hetzner Object Storage (S3-kompatibel), privater Bucket	Falkenstein	Pitchdecks/Media privat , nur signierte URLs (§12.6)
WAF/DDoS	Myra Security (BSI-C5, §8a BSIG-qualifiziert, DE)	DE	Deutsche WAF vor der App — starkes Argument im Partner-Audit. Alternative: Cloudflare (dann §17.5 TIA)
E-Mail	Brevo oder Mailjet (FR, EU) bzw. Scaleway TEM	EU	ersetzt den <code>consoleEmailAdapter</code> ; nicht SES/Resend/Postmark (US)
Scheduler	systemd-Timer bzw. Kamal-Cron → <code>POST /api/cron/reminders</code> mit <code>CRON_SECRET</code>	DE	ersetzt CF Cron Triggers
Secrets	Infisical (self-hosted) oder HashiCorp Vault, Injection zur Deploy-Zeit	DE	§10.3
Monitoring	Grafana + Loki + Prometheus (self-hosted) oder Grafana Cloud EU	DE/EU	§16
BI	Metabase self-hosted gegen DWH	DE	§15.7

Netz-Zonierung



Vorteile

- Vollständige **EU-/DE-Vertragskette**, im Compliance-Fragebogen in einem Satz beantwortbar.
- **Argon2id** möglich (F-09 auflösbar) — direkter Sicherheitsgewinn in der Authentifizierung.
- Echte **Audit-/Forensikfähigkeit**: persistente Logs, `pgaudit`, unbegrenzte Job-Laufzeit.
- **RLS in Postgres** als zweite Autorisierungsschicht praktikabel (§7.4).
- Freie ETL-/dbt-Läufe für das DWH, keine Runtime-Limits.
- Kein Prisma-WASM-Bundle-Workaround mehr nötig (`compilerBuild = "small"` kann entfallen).
- Kosten planbar und niedrig; kein anfrage- oder egress-basiertes Preisrisiko.

Nachteile / Risiken

Nachteil	Gegenmaßnahme
Ops-Verantwortung für App-Nodes (Patches, Kernel, Container)	Unattended-Upgrades, Image-Rebuild-Pipeline, wöchentliches Patch-Fenster; DB bleibt managed
Kein globales Edge-CDN → höhere Latenz außerhalb Europas	Nutzerbasis ist DE/EU; statische Assets über WAF-CDN cachen
Single-Provider-Risiko Hetzner	Container + IaC sind portabel (Exit in <1 Woche zu Scaleway/IONOS); DB bei separatem Anbieter
Migrationsaufwand aus Szenario B	§5.7, geschätzt 5–8 Personentage

Kostenschätzung (Produktion, monatlich, netto)

Posten	Betrag
2 × CPX31 App-Nodes	~ 32 €
Hetzner Load Balancer + Private Network	~ 7 €
Managed Postgres HA (OLTP, ~4 GB RAM, PITR)	70–120 €
DWH-Postgres (klein)	20–30 €
Object Storage (250 GB)	~ 6 €
Myra WAF (Einstiegspaket)	50–150 €

Posten	Betrag
E-Mail (EU, < 20 k Mails)	0–25 €
Monitoring/BI (self-hosted, 1 × CX22)	~ 5 €
Summe	~ 190–375 €/Monat

Dazu ~4–6 Ops-Stunden/Monat im Regelbetrieb.

5.4 Szenario B — Cloudflare Workers + Neon (Status quo, gehärtet)

Aufbau: wie heute, aber produktionsreif gemacht: eigener Prod-Worker mit Custom Domain (`app.lovedis.de`), Neon-Projekt **Region Frankfurt**, Cloudflare **Data Localization Suite** (EU-Region für Traffic und Logs), WAF-Regeln + Rate-Limiting-Rules, Cron Triggers, Workers Logs mit maximaler Retention, Hyperdrive optional.

Vorteile

- **Kein Migrationsaufwand** — Repo ist bereits darauf ausgelegt (`wrangler.jsonc`, `open-next.config.ts`, Runtime-Switch in `src/lib/prisma.ts`).
- Beste DDoS-Absorption am Markt, global niedrige Latenz, sehr niedrige Grundkosten (~20–60 €/Monat ohne DLS; mit DLS deutlich mehr, da Enterprise).
- Keine Server-Patches, keine SSH-Angriffsfläche, keine Kernel-CVEs.
- Isolation ist bereits gut umgesetzt (getrennter Account, keine Routes).

Nachteile (die die Empfehlung gegen B begründen)

Nachteil	Auswirkung	Milderbar?
US-Vertragskette (Cloudflare + Neon/Databricks) → CLOUD Act	Compliance-Argumentationslast bei jedem Partner-Audit; TIA nötig	Nur teilweise (DLS, SCC, Verschlüsselung)
Kein Argon2id (Workers-Runtime + CPU-Limit)	Passwort-Hashing bleibt auf bcryptjs-Niveau	Nein
Keine persistenten Prozesse, CPU-/Bundle-Limits	ETL/DWH-Jobs, große Exporte, Forensik-Tooling nur mit Workarounds	Teilweise (Queues, Containers)
Eingeschränkte Log-Retention & Forensik	Incident Response erschwert; Audit-Log muss vollständig in die DB	Teilweise (Logpush → R2/extern)
DLS ist Enterprise-Add-on	Kostenvorteil von B verschwindet weitgehend, sobald man EU-Residenz wirklich will	Nein
Prisma-WASM-Bundle am Limit	Bereits Workarounds im Repo; jedes Schema-Wachstum ist ein Risiko	Teilweise
RLS/ <code>pgaudit</code> /Extensions bei Neon eingeschränkt	Defense-in-Depth in der DB schwächer	Teilweise

Fazit zu B: Betreibbar, und für einen internen Pilot mit wenigen Partnern völlig in Ordnung. Aber B kann zwei Dinge **strukturell** nicht liefern, die für diese Datenklasse zählen: US-Unabhängigkeit und Argon2id. Wenn B gewählt wird, muss das eine dokumentierte Management-Entscheidung sein (§20, Entscheidung 1).

5.5 Szenario C — AWS `eu-central-1` (Frankfurt)

Aufbau: ECS Fargate (oder EKS) für die Next.js-Container, **Aurora PostgreSQL** Serverless v2 Multi-AZ, Secrets Manager, KMS mit eigenen Keys, WAF + Shield Advanced, CloudTrail + GuardDuty + Security Hub, S3 für Uploads, Redshift/Athena optional als DWH.

Vorteile: Der reifste Compliance- und Security-Werkzeugkasten (CloudTrail-Audit, GuardDuty, KMS-Key-Hoheit, IAM-Feingranularität), Multi-AZ-HA out of the box, unbegrenzte Skalierung, alle gängigen Zertifizierungen (C5, ISO 27001, SOC 2).

Nachteile: US-Mutterkonzern → **dieselbe CLOUD-Act-Diskussion wie B**, nur teurer. Deutlich höhere Komplexität (VPC, IAM, Terraform) und Kosten (realistisch 400–900 €/Monat für dieselbe Last), erhebliches Fehlkonfigurationsrisiko (offene S3-Buckets, zu weite IAM-Policies) — bei einem kleinen Team ist die *Komplexität selbst* ein Sicherheitsrisiko.

Wann trotzdem C: Wenn ein Großpartner explizit AWS-C5-Testate in der Lieferkette fordert oder die Plattform auf > 10 000 Nutzer skaliert.

5.6 Entscheidungsmatrix

Punkte 1–5 (5 = am besten), gewichtet nach §5.1.

Kriterium	Gew.	A: Hetzner/EU	B: Cloudflare	C: AWS FRA	F: Scaleway
K1 Souveränität EU/DE	25 %	5	2	2	5
K2 Sicherheitsfähigkeit	20 %	4	2	5	4
K3 Betriebsaufwand	15 %	3	5	2	3
K4 Kosten (TCO)	10 %	4	5	1	4
K5 Stack-Passung	15 %	5	3	4	5
K6 DWH-Tauglichkeit	10 %	5	2	5	4
K7 HA & Exit	5 %	3	3	5	3
Gewichtete Summe		4,30	2,95	3,25	4,20

A gewinnt vor allem über K1, K5 und K6. B gewinnt nur bei Betriebsaufwand und Kosten — beides Kriterien, die zusammen 25 % wiegen und den Souveränitäts-/Krypto-Nachteil nicht aufheben. **Scaleway (F) liegt praktisch gleichauf mit A** und ist die naheliegende Alternative, wenn eine französische statt deutschen Vertragskette akzeptabel ist oder Managed-Kubernetes gewünscht wird.

5.7 Migrationspfad B → A (5–8 Personentage)

Schritt	Inhalt	Aufwand	Risiko
1	IaC-Grundgerüst (Terraform: Hetzner-Projekt, Netz, LB, Nodes, Firewall, WireGuard-Bastion)	1,5 d	niedrig
2	Managed Postgres bereitstellen, TLS + IP-Allowlist + DB-Rollen (§7.2)	0,5 d	niedrig
3	<code>Dockerfile</code> (multi-stage, <code>output: "standalone"</code> , non-root) + Kamal-Deploy	1 d	niedrig
4	<code>src/lib/prisma.ts</code> vereinfachen: Neon-Zweig entfällt, nur <code>PrismaPg</code> ; <code>compilerBuild = "small"</code> und <code>outputFileTracingIncludes</code> entfernen	0,25 d	niedrig
5	Argon2id-Migration mit Lazy-Rehash (§9.2)	0,5 d	mittel — sorgfältig testen
6	Secrets nach Infisical/Vault überführen, <code>wrangler secret</code> ablösen	0,5 d	mittel — Vollständigkeit prüfen
7	Datenmigration Neon → Zielsystem (<code>pg_dump -Fc</code> / logische Replikation für Near-Zero-Downtime)	0,5 d	mittel
8	WAF (Myra) vorschalten, Origin-Allowlist, DNS-Umstellung mit niedriger TTL	0,5 d	mittel
9	Cron: CF Trigger → systemd-Timer; <code>CRON_SECRET</code> neu setzen	0,25 d	niedrig

Schritt	Inhalt	Aufwand	Risiko
10	Monitoring/Loki/Alerting, Smoke-Tests (<code>/api/health</code> , Login, Matrix, Booking-Flow)	1 d	niedrig
11	Cutover + 14 Tage Cloudflare-Worker als Rollback-Ziel vorhalten	0,5 d	niedrig

Wichtig: Die Live-Homepage `lovedis.de` bleibt gemäß `docs/deployment-plan-mara.md` unberührt. Die Plattform kommt auf `app.lovedis.de` — nur ein **zusätzlicher** DNS-Record, kein Eingriff in Apex-/`www`-Routing.

6. Ziel-Architektur im Detail

6.1 Umgebungen

Umgebung	Zweck	Daten	Zugang	DB
local	Entwicklung	Seed-Daten (<code>embedded-postgres</code> , Port 5433)	Entwickler	lokal
staging	Integration, UAT	anonymisierte Kopie der Prod-Struktur, nie echte K3-Daten	Team + WireGuard	eigene Instanz
production	Betrieb	echte Daten	Nutzer über WAF; Admin nur mit MFA	HA + PITR

Regeln, die nicht verhandelbar sind:

1. Kein Produktionsdatensatz gelangt unmaskiert nach staging oder local (§15.6 Maskierungsskript).
2. Kein Secret ist zwischen Umgebungen geteilt — auch nicht „nur der DB-User“.
3. `prisma db push` ist in Produktion verboten; dort ausschließlich versionierte `prisma migrate deploy` (§7.7).
4. Deploys nach Produktion nur aus `main` über die Pipeline, niemals von einem Laptop.

6.2 Vertrauenszonen

Zone	Inhalt	Eintritt erlaubt aus
Z0 Internet	Unvertrauenswürdig	—
Z1 Edge	WAF, DDoS, TLS, Bot-Management	Z0 (443)
Z2 Anwendung	App-Nodes, Caddy, Next.js	nur Z1 (IP-Allowlist + Shared Secret)
Z3 Daten	OLTP-Postgres, Object Storage	nur Z2 (privates Netz, TLS, DB-Allowlist)
Z4 Analytics	DWH-Postgres, dbt-Runner, Metabase	nur Z2/Z5, niemals aus Z0
Z5 Management	WireGuard-Bastion, Secrets-Manager, Monitoring	nur MFA-Admins

Kritische Regel: Z3 und Z4 haben **keine öffentliche IP**. Ein direkter `psql`-Versuch aus dem Internet muss auf Netzebene scheitern, nicht erst an der Passwortprüfung.

7. Datenbank-Sicherheit & -Betrieb

7.1 Grundhärtung PostgreSQL 17

Maßnahme	Einstellung	Zweck
Transportverschlüsselung erzwingen	<code>ssl = on</code> , <code>sslmode=verify-full</code> im Client-DSN inkl. <code>sslrootcert</code>	Verhindert MITM und Fehlleitung auf falschen Host
Netzzugang	<code>listen_addresses</code> = private IP; <code>pg_hba.conf</code> nur App-Subnetz + Bastion	Kein Internet-Exposure
Auth-Verfahren	<code>password_encryption = scram-sha-256</code> , kein <code>md5</code>	Moderner Challenge-Response
Extension-Kontrolle	nur <code>pgcrypto</code> , <code>pgaudit</code> , <code>pg_stat_statements</code> ; <code>trust</code> - Extensions verbieten	Angriffsfläche
Logging	<code>log_connections</code> , <code>log_disconnections</code> , <code>log_statement = 'ddl'</code> , <code>pgaudit.log = 'write,ddl,role'</code>	Nachvollziehbarkeit (§11.5)
Zeitlimits	<code>statement_timeout = 30s</code> (App-Rolle), <code>idle_in_transaction_session_timeout = 60s</code>	DoS-Schutz, kein Lock-Stau
Storage-Verschlüsselung	Encryption at Rest des Anbieters (AES-256)	Physischer Diebstahl / Entsorgung
Superuser	Anwendung läuft nie als Superuser	Least Privilege

7.2 Datenbank-Rollenmodell

Statt eines Allzweck-Users vier Rollen — heute nutzt die Anwendung einen einzigen DSN, das ist zu viel Recht in einer Hand:

Rolle	Rechte	Nutzung
<code>lovedis_app</code>	<code>SELECT/INSERT/UPDATE/DELETE</code> auf Anwendungstabellen, kein DDL , kein <code>TRUNCATE</code>	Laufzeit-DSN (<code>DATABASE_URL</code>)
<code>lovedis_migrate</code>	zusätzlich DDL	nur in der CI-Migrationsstufe
<code>lovedis_readonly</code>	<code>SELECT</code> , <code>default_transaction_read_only = on</code>	ETL-Extraktion ins DWH (§15.3)
<code>lovedis_audit</code>	<code>INSERT</code> auf <code>AuditEvent</code> , kein <code>UPDATE</code> / <code>DELETE</code>	Append-only-Audit (§11.5)

```
-- Beispiel: Laufzeitrolle ohne DDL und ohne Schema-Änderungsrechte
CREATE ROLE lovedis_app LOGIN PASSWORD 'app_pw';
REVOKE ALL ON SCHEMA public FROM PUBLIC;
GRANT USAGE ON SCHEMA public TO lovedis_app;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO lovedis_app;
ALTER DEFAULT PRIVILEGES IN SCHEMA public
    GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES TO lovedis_app;
ALTER ROLE lovedis_app SET statement_timeout = '30s';

-- Audit-Rolle: nur anfügen, nie ändern oder löschen
CREATE ROLE lovedis_audit LOGIN PASSWORD 'audit_pw';
GRANT INSERT ON "AuditEvent" TO lovedis_audit;
```

Warum das zählt: Eine SQL-Injection oder ein kompromittierter App-Prozess kann dann **keine Tabelle löschen** und **kein Audit-Log manipulieren**.

7.3 Integritäts-Constraints auf DB-Ebene

Prisma validiert in der Applikation; die Datenbank sollte dieselben Invarianten unabhängig durchsetzen (Defense in Depth gegen Tampering, §4.2):

```
-- Score-Werte sind laut Domänenlogik 0-5
ALTER TABLE "Score" ADD CONSTRAINT score_value_range CHECK (value BETWEEN 0 AND 5);

-- Credit-Invariante aus dem Schema-Kommentar auch in der DB verankern
ALTER TABLE "CreditAccount" ADD CONSTRAINT credit_balance_split
CHECK (balance = "fixBalance" + "flexBalance");
ALTER TABLE "CreditAccount" ADD CONSTRAINT credit_no_negative
CHECK ("fixBalance" >= 0 AND "flexBalance" >= 0);

-- Genau eine Ziel-Referenz je Marktplatz-Buchung
ALTER TABLE "MarketplaceBooking" ADD CONSTRAINT booking_single_target CHECK (
  (("programId" IS NOT NULL)::int + ("mentorId" IS NOT NULL)::int
  + ("offeringId" IS NOT NULL)::int) = 1
);

-- Credit-Transaktionen sind unveränderlich: UPDATE/DELETE per Trigger verbieten
CREATE OR REPLACE FUNCTION forbid_mutation() RETURNS trigger AS $$
BEGIN RAISE EXCEPTION 'append-only table'; END; $$ LANGUAGE plpgsql;

CREATE TRIGGER credit_tx_append_only
BEFORE UPDATE OR DELETE ON "CreditTransaction"
FOR EACH ROW EXECUTE FUNCTION forbid_mutation();
```

Hinweis: Storno wird laut [docs/mara-implementation-notes.md](#) bereits korrekt als **positive ADJUSTMENT** -Gegenbuchung umgesetzt, nicht als Löschung. Der Append-only-Trigger passt also zur bestehenden Logik und macht sie erzwungen statt nur konventionell.

7.4 Row-Level Security als zweite Autorisierungsschicht

Problem (F-12): Die gesamte Autorisierung hängt an `requireRole()` -Aufrufen in Server Actions und Page-Komponenten. Eine neue Action ohne Guard = stiller Datenabfluss von K3-Daten. Bei aktuell ~40 Modellen und wachsendem Team ist das eine Frage der Zeit, nicht des Willens.

Lösung: RLS für die drei kritischsten Tabellen. Die App setzt pro Transaktion den Kontext, die DB erzwingt die Sichtbarkeit:

```

ALTER TABLE "PartnerStartupReview" ENABLE ROW LEVEL SECURITY;
ALTER TABLE "PartnerStartupReview" FORCE ROW LEVEL SECURITY;

-- Team (ADMIN/MEMBER) sieht alles
CREATE POLICY psr_team ON "PartnerStartupReview" FOR ALL
  USING (current_setting('app.role', true) IN ('ADMIN','MEMBER'));

-- Ein Partner sieht ausschließlich SEINE eigenen Verdikte
CREATE POLICY psr_own_partner ON "PartnerStartupReview" FOR ALL
  USING (current_setting('app.role', true) = 'BUSINESS_PARTNER'
    AND "partnerId" = current_setting('app.user_id', true));

-- Match-Matrix ist rein intern (Schema-Kommentar sagt: nicht für Partner/Startups/Investoren)
ALTER TABLE "PartnerStartupMatch" ENABLE ROW LEVEL SECURITY;
ALTER TABLE "PartnerStartupMatch" FORCE ROW LEVEL SECURITY;
CREATE POLICY psm_team_only ON "PartnerStartupMatch" FOR ALL
  USING (current_setting('app.role', true) IN ('ADMIN','MEMBER'));

```

Kontextsetzung in der Anwendung — als Prisma-Client-Extension, damit sie nicht vergessen werden kann:

```

// src/lib/prisma-rls.ts
export function withUserContext(session: Session) {
  return prisma.$extends({
    query: {
      $allModels: {
        async $allOperations({ args, query }) {
          return prisma.$transaction(async (tx) => {
            await tx.$executeRaw`SELECT set_config('app.user_id', ${session.user.id}, true)`;
            await tx.$executeRaw`SELECT set_config('app.role', ${session.user.role}, true)`;
            return query(args);
          });
        },
      },
    },
  });
}

```

Der dritte Parameter `true` bei `set_config` macht die Einstellung **transaktionslokal** — das ist bei Connection Pooling zwingend, sonst leckt der Kontext in fremde Requests.

Migrationsreihenfolge: RLS zuerst mit `USING (true)` -Policies aktivieren und im Log mitschreiben, welche Queries betroffen wären; erst danach scharf stellen. Sonst brechen Hintergrundjobs unangekündigt.

7.5 Verschlüsselung sensibler Felder (F-11)

Storage-Encryption schützt gegen gestohlene Datenträger — **nicht** gegen einen Dump durch einen legitimen DB-Zugang oder ein geleaktes Backup. Für die höchstsensiblen Freitextfelder deshalb Verschlüsselung **in der Anwendung** (envelope encryption, Schlüssel im Secrets-Manager):

Feld	Verfahren	Begründung
<code>Message.body</code>	AES-256-GCM, App-seitig	Vertrauliche 1:1-Kommunikation, nie durchsuchbar nötig
<code>Contact.email</code> , <code>.phone</code> , <code>.notes</code>	AES-256-GCM + blind index (HMAC-SHA256) für Suche	PII Dritter, muss auffindbar bleiben
<code>PartnerStartupReview.note</code>	AES-256-GCM	K3-Freitext-Votum

Feld	Verfahren	Begründung
<code>Attachment.url</code> (privater Bucket-Key)	Klartext ok, Zugriff über signierte URL	§12.6
<code>User.passwordHash</code>	nicht verschlüsseln — Argon2id-Hash	Hash ≠ Verschlüsselung

```
// src/lib/field-crypto.ts – AES-256-GCM mit Key-Versionierung für Rotation
import { createCipheriv, createDecipheriv, randomBytes, createHmac } from "node:crypto";

const KEYS: Record<string, Buffer> = {
  v1: Buffer.from(process.env.FIELD_KEY_V1!, "base64"),
};
const CURRENT = "v1";

export function encryptField(plain: string): string {
  const iv = randomBytes(12);
  const cipher = createCipheriv("aes-256-gcm", KEYS[CURRENT], iv);
  const ct = Buffer.concat([cipher.update(plain, "utf8"), cipher.final()]);
  // Präfix mit Key-Version → Rotation ohne Rückwärts-Migration
  return [CURRENT, iv.toString("base64"), cipher.getAuthTag().toString("base64"),
    ct.toString("base64")].join(":");
}

export function decryptField(stored: string): string {
  const [version, iv, tag, ct] = stored.split(":");
  const decipher = createDecipheriv("aes-256-gcm", KEYS[version], Buffer.from(iv, "base64"));
  decipher.setAuthTag(Buffer.from(tag, "base64"));
  return Buffer.concat([decipher.update(Buffer.from(ct, "base64")),
    decipher.final()]).toString("utf8");
}

/** Deterministischer Blind Index für Gleichheitssuche auf verschlüsselten Feldern. */
export function blindIndex(value: string): string {
  return createHmac("sha256", KEYS[CURRENT]).update(value.trim().toLowerCase()).digest("hex");
}
```

Schema-Ergänzung für die Suchbarkeit:

```
model Contact {
  // ...
  email          String? // verschlüsselt (v1:iv:tag:ct)
  emailBlindIdx  String? // HMAC für Gleichheitssuche
  @@index([emailBlindIdx])
}
```

Bewusste Grenze: Bereichs- und Teilstring-Suche über verschlüsselte Felder ist damit nicht mehr möglich. Das ist bei `Contact.email` akzeptabel (Suche erfolgt exakt), bei `Message.body` ebenfalls (es gibt keine Volltextsuche über Nachrichten). Wenn später eine Nachrichtensuche gewünscht wird, muss sie clientseitig oder über ein separates, geschütztes Index-Verfahren gelöst werden — nicht durch Rücknahme der Verschlüsselung.

7.6 Connection Pooling & Query-Hygiene

- **PgBouncer** (Transaction Pooling) bzw. der Pooler des Anbieters; `connection_limit` in der DSN passend zur Node-Zahl setzen, damit `max_connections` nicht erschöpft wird.
- Prisma-Singleton bleibt wie in `src/lib/prisma.ts` (Global-Cache in Dev, damit HMR keine Verbindungen leakt).

- **Query-Timeouts** auf der App-Rolle (§7.1) verhindern, dass eine teure Matrix- oder Report-Query die Instanz blockiert (DoS, §4.2).
- `pg_stat_statements` aktivieren und die 20 teuersten Queries monatlich reviewen. Die Match-Matrix (`Partner-StartupMatch` × `Startup` × `PartnerCompany`) und die Reports sind die wahrscheinlichsten Kandidaten.
- Prisma-Logging: `query` -Log in Produktion aus — es schreibt Parameterwerte und damit K3-Daten ins Log.

7.7 Schema-Migrationen (Ablösung von `prisma db push`)

Heute wird laut `README.md` und `package.json` mit `prisma db push` gearbeitet. Für Produktion ist das nicht tragfähig: kein Änderungsnachweis, kein Rollback, Gefahr von Datenverlust bei Spaltenumbenennungen.

Umgebung	Verfahren
local	<code>prisma db push</code> (schnelle Iteration) bleibt erlaubt
staging	<code>prisma migrate deploy</code> aus dem Migrations-Ordner
production	nur <code>prisma migrate deploy</code> in der Pipeline, mit <code>lovedis_migrate</code> -Rolle

Zusätzliche Schutzmaßnahmen:

```
// prisma/seed.ts – harter Riegel gegen versehentliches Seeden in Produktion (F-01)
if (process.env.NODE_ENV === "production" || process.env.ALLOW_SEED !== "true") {
  throw new Error(
    "Seed in Produktion blockiert. Demo-Accounts und das Seed-Passwort dürfen " +
    "niemals in einer Produktionsdatenbank existieren."
  );
}
```

- Vor jeder Prod-Migration: automatischer Snapshot + Ausgabe des Ziel-Hosts zur Bestätigung (das Risiko „`DATA-BASE_URL` zeigt auf die falsche DB" ist in `docs/deployment-plan-mara.md` §10 bereits als Risiko erkannt).
- Destruktive Migrationen (DROP COLUMN/TABLE) benötigen ein Zwei-Augen-Approval im PR.
- Expand/Contract-Muster: erst Spalte hinzufügen + doppelt schreiben, dann lesen umstellen, dann alte Spalte in einer späteren Migration entfernen.

8. Backup, PITR & Disaster Recovery

8.1 Backup-Strategie (3-2-1)

Ebene	Verfahren	Frequenz	Aufbewahrung	Ort
PITR (WAL)	Continuous Archiving des Anbieters	fortlaufend	14 Tage	DB-Region
Vollbackup	<code>pg_dump -Fc</code> , automatisiert	täglich 03:00	30 Tage	anderer Anbieter/Region
Monatsarchiv	verschlüsseltes Dump-Archiv	monatlich	12 Monate	Object Storage, Objekt-Lock
Offline-Kopie	verschlüsselte Kopie außerhalb der Cloud	quartalsweise	1 Jahr	Safe / getrennter Account
Objektspeicher	Versionierung + Lifecycle	fortlaufend	90 Tage Versionen	DE
Secrets	verschlüsseltes Export-Backup des Secret-Stores	monatlich	6 Monate	getrennter Tresor

Zwei Regeln, die Ransomware-Resistenz erzeugen:

1. Backups liegen **nicht im selben Account** wie die Produktionsdatenbank. Ein kompromittierter Prod-Zugang darf die Backups nicht löschen können.
2. Monatsarchive mit **Object Lock / WORM** (unveränderlich für die Retention-Dauer).
3. Backups sind **client-seitig verschlüsselt** (z. B. `age` /GPG), sodass auch der Storage-Anbieter sie nicht lesen kann.

8.2 RPO/RTO-Ziele

Szenario	RPO	RTO	Verfahren
App-Node-Ausfall	0	< 5 min	LB nimmt Node aus Rotation, zweiter Node trägt Last
DB-Primary-Ausfall	< 1 min	< 15 min	Managed HA-Failover auf Standby
Fehlerhafte Migration / Datenverlust	< 5 min	< 4 h	PITR auf Zeitpunkt vor Migration
Versehentliche Löschung einzelner Zeilen	< 5 min	< 2 h	PITR-Klon → gezieltes Zurückkopieren
Ransomware / Totalkompromittierung	< 24 h	< 24 h	Neuaufbau via IaC + Monatsarchiv, alle Secrets rotieren
Providerausfall (Region)	< 24 h	< 48 h	Wiederherstellung bei Alternativanbieter aus Off-Site-Dump

8.3 Restore-Tests (behebt F-15)

Ein Backup, das nie zurückgespielt wurde, ist eine Annahme, kein Backup.

Test	Frequenz	Nachweis
Vollrestore in eine Wegwerf-DB + <code>/api/health</code> + Login-Smoke-Test	monatlich	Protokoll mit Dauer, Datenstand, Prüfer
PITR auf einen Zeitpunkt „vor 6 Stunden“	quartalsweise	Protokoll
Vollständige DR-Übung (Neuaufbau aus IaC, Alternativanbieter)	jährlich	Bericht + Nachbesserungsliste

Verifikationsabfragen nach jedem Restore (fachliche Plausibilität, nicht nur „DB startet“):

```
SELECT count(*) FROM "User" WHERE "isActive";
SELECT count(*) FROM "PartnerStartupMatch";
-- Credit-Invariante: gecachte Salden müssen zum Ledger passen
SELECT a.id, a.balance, COALESCE(SUM(t.amount), 0) AS ledger
FROM "CreditAccount" a LEFT JOIN "CreditTransaction" t ON t."accountId" = a.id
GROUP BY a.id, a.balance HAVING a.balance <> COALESCE(SUM(t.amount), 0);
-- Erwartung: 0 Zeilen. Jede Zeile hier ist ein Integritätsschaden.
```

9. Identität, Authentifizierung & Sessions

9.1 Zielbild

Aspekt	Heute	Ziel
Hashing	<code>bcryptjs</code>	Argon2id (Node-Runtime, Szenario A)
MFA	keine	TOTP-Pflicht für <code>ADMIN</code> / <code>MEMBER</code> , optional für alle
Rate-Limit	keiner	IP + Konto, progressives Backoff
Reset	keiner	Token-basiert, 30 min gültig, Single-Use
Session	JWT, Default 30 Tage	JWT 15 min + rotierendes Refresh, serverseitig widerrufbar
Passwort-Policy	keine	≥ 12 Zeichen, HIBP-Breach-Check
Lockout	keiner	10 Fehlversuche → 15 min Sperre + Benachrichtigung

9.2 Argon2id-Migration (F-09)

Empfohlene Parameter (OWASP Password Storage Cheat Sheet): `m = 19 MiB` , `t = 2` , `p = 1` .

```
// src/lib/password.ts
import { hash, verify } from "@node-rs/argon2";

const ARGON2_OPTS = {
  memoryCost: 19456, // 19 MiB
  timeCost: 2,
  parallelism: 1,
  algorithm: 2,      // Argon2id
} as const;

export function hashPassword(plain: string): Promise<string> {
  return hash(plain, ARGON2_OPTS);
}

export async function verifyPassword(plain: string, stored: string): Promise<boolean> {
  // Legacy-bcrypt-Hashes ($2a$/2b$/2y$) weiter akzeptieren, damit niemand ausgesperrt wird.
  if (stored.startsWith("$2")) {
    const bcrypt = await import("bcryptjs");
    return bcrypt.compare(plain, stored);
  }
  return verify(stored, plain, ARGON2_OPTS);
}

export function needsRehash(stored: string): boolean {
  return stored.startsWith("$2");
}
```

Lazy-Rehash beim nächsten erfolgreichen Login — es gibt keinen Zwangs-Reset für alle:

```
// src/auth.ts (Auszug, ersetzt den bcrypt-Vergleich)
const user = await prisma.user.findUnique({
  where: { email: parsed.data.email.toLowerCase() },
  // F-16: nur benötigte Felder laden
  select: { id: true, email: true, name: true, role: true,
    passwordHash: true, isActive: true },
});
if (!user || !user.isActive) return null;

const ok = await verifyPassword(parsed.data.password, user.passwordHash);
if (!ok) return null;

if (needsRehash(user.passwordHash)) {
  await prisma.user.update({
    where: { id: user.id },
    data: { passwordHash: await hashPassword(parsed.data.password) },
  });
}
```

Wichtiger Nebeneffekt: Dieses Muster funktioniert **nur** auf einer Node-Runtime. `@node-rs/argon2` ist ein native Adon und läuft nicht in Workers. Das ist der konkrete Sicherheitsgewinn aus Szenario A (§1.2).

Timing-Angriffe: Bei nicht existierendem Benutzer heute sofortiges `return null` — die Antwortzeit verrät, ob eine E-Mail registriert ist (User Enumeration). Gegenmaßnahme: immer gegen einen Dummy-Hash verifizieren, bevor `null` zurückgegeben wird.

9.3 Multi-Faktor-Authentifizierung (F-03)

Pflicht für `ADMIN` und `MEMBER` — diese Rollen sehen laut `requireScoutModule` / `requireTeam` die Match-Matrix und alle Bewertungen (K3). Optional, aber empfohlen für `BUSINESS_PARTNER`.

```
model User {
  // ...
  totpSecret      String?    // AES-256-GCM verschlüsselt (§7.5)
  totpConfirmedAt DateTime?
  totpRecoveryCodes String[] @default([]) // Argon2id-Hashes, Single-Use
  mfaRequired     Boolean    @default(false) // für ADMIN/MEMBER true
}
```

- Verfahren: **TOTP (RFC 6238)**, 30-s-Fenster, ± 1 Schritt Toleranz, Secret AES-verschlüsselt.
- 10 Recovery-Codes, gehasht gespeichert, Single-Use, bei Nutzung Alarm ins Monitoring.
- Replay-Schutz: verbrauchten Zeitschritt je Nutzer kurz sperren.
- Reihenfolge im Login-Flow: Passwort → *dann* TOTP; das JWT wird erst nach dem zweiten Faktor ausgestellt (`amr: ["pwd", "otp"]` im Token vermerken).
- **Optional stärker:** WebAuthn/Passkeys für Admins. Höherer Implementierungsaufwand, aber phishing-resistent — sinnvoll als P2.

9.4 Rate-Limiting & Lockout (F-02)

Drei Ebenen, weil jede einzelne umgehbar ist:

Ebene	Regel	Umsetzung
Edge/WAF	10 <code>POST /api/auth/callback/credentials</code> pro IP / 5 min	Myra bzw. Cloudflare Rate-Limiting-Rule

Ebene	Regel	Umsetzung
Anwendung	5 Fehlversuche pro Konto / 15 min → 15 min Sperre	DB-Zähler oder Valkey/Redis
Konto	10 Fehlversuche kumulativ → temporäre Sperre + E-Mail an den Nutzer	LoginAttempt -Modell

```

model LoginAttempt {
  id          String   @id @default(cuid())
  email       String   // normalisiert (lowercase)
  ipHash      String   // HMAC der IP – kein Klartext–Personenbezug (§17.2)
  successful   Boolean
  createdAt   DateTime @default(now())

  @@index([email, createdAt])
  @@index([ipHash, createdAt])
}

model User {
  // ...
  failedLoginCount Int      @default(0)
  lockedUntil      DateTime?
}

```

Wichtig: Die Sperre wird **auf das Konto** angewendet, aber die Fehlermeldung bleibt generisch („E-Mail oder Passwort ist falsch“) — sonst entsteht wieder User Enumeration. Ein gesperrtes Konto erhält eine E-Mail-Benachrichtigung, damit der legitime Nutzer den Angriff bemerkt.

9.5 Session-Härtung (F-08, F-19)

JWT-Sessions sind schnell und Edge-tauglich (deshalb funktioniert `middleware.ts` ohne DB), haben aber einen strukturellen Nachteil: **ein ausgestelltes Token lässt sich nicht widerrufen**. Wird ein Laptop gestohlen oder ein Konto kompromittiert, gilt das Token bis zum Ablauf — im Default 30 Tage.

Zwei Optionen:

Option	Vorgehen	Bewertung
B1 (empfohlen)	JWT kurzlebig (15 min) + <code>sessionsValidFrom</code> -Spalte am User. <code>requireAuth()</code> liest den User bereits aus der DB — dort zusätzlich prüfen, ob <code>token.iat >= user.sessionsValidFrom</code> . Logout-überall = Spalte auf <code>now()</code> setzen.	Minimaler Eingriff, nutzt den existierenden DB-Roundtrip in <code>requireAuth</code> , sofort wirksam
B2	Umstieg auf DB-Sessions (<code>Session</code> -Tabelle)	Vollständige Kontrolle, aber <code>middleware.ts</code> verliert die Edge-Tauglichkeit

```

model User {
  // ...
  // Alle vor diesem Zeitpunkt ausgestellten JWTs gelten als ungültig.
  // Setzen bei: Passwortwechsel, MFA-Reset, Rollenwechsel, "überall abmelden",
  // Deaktivierung und Sicherheitsvorfall.
  sessionsValidFrom DateTime @default(now())
}

```

```
// src/lib/auth-guards.ts – Erweiterung von requireAuth()
const user = await prisma.user.findUnique({
  where: { id: session.user.id },
  select: { id: true, isActive: true, role: true, sessionsValidFrom: true },
});
if (!user || !user.isActive) redirect("/api/session-clear");

// Serverseitiger Widerruf: Token, das vor dem Cut-off ausgestellt wurde, verfällt.
const issuedAt = session.issuedAt; // aus dem jwt-Callback in den Session-Callback durchreichen
if (issuedAt && issuedAt < user.sessionsValidFrom) redirect("/api/session-clear");
```

Weitere Session-Einstellungen:

```
// src/auth.config.ts
session: { strategy: "jwt", maxAge: 60 * 60 * 8, updateAge: 60 * 15 }, // 8 h Arbeitstag
cookies: {
  sessionToken: {
    // __Host-Präfix: erzwingt Secure, Path=/ und verbietet Domain-Attribut
    // → kein Setzen durch Subdomains, kein Cookie-Tossing
    name: "__Host-authjs.session-token",
    options: { httpOnly: true, sameSite: "lax", path: "/", secure: true },
  },
},
```

`sameSite: "lax"` ist nötig, weil der OAuth-/Callback-Redirect sonst bricht; `strict` würde den Login-Flow beschädigen. Der CSRF-Schutz kommt bei Server Actions ohnehin aus dem Origin-Check von Next.js.

9.6 Passwort-Reset (F-04)

Ohne Reset-Flow entstehen in der Praxis die schlimmsten Passwort-Praktiken: Admins setzen Passwörter manuell und schicken sie per E-Mail oder Slack.

```
model PasswordResetToken {
  id          String    @id @default(cuid())
  userId      String
  user        User      @relation(fields: [userId], references: [id], onDelete: Cascade)
  tokenHash   String    @unique // SHA-256 des Tokens – Klartext nur in der E-Mail
  expiresAt   DateTime
  usedAt      DateTime?
  createdAt   DateTime  @default(now())

  @@index([userId])
  @@index([expiresAt])
}
```

Anforderungen an den Flow:

1. Token = 32 Byte aus `crypto.randomBytes`, **nur der Hash** wird gespeichert.
2. Gültigkeit **30 Minuten**, strikt Single-Use (`usedAt`).
3. Die Antwort ist **immer** identisch („Falls ein Konto existiert, wurde eine E-Mail versendet“) — keine Enumeration.
4. Rate-Limit: 3 Anfragen pro E-Mail und Stunde.
5. Bei Erfolg: alte Tokens invalidieren, `sessionsValidFrom = now()` (§9.5) → alle Sessions fliegen raus, und eine Bestätigungs-E-Mail an den Nutzer („Ihr Passwort wurde geändert“).
6. Ein Reset **hebt MFA nicht auf**. Andernfalls wäre der Reset-Flow ein MFA-Bypass.

9.7 Passwort-Policy & Breach-Check (F-14)

Regel	Wert	Begründung
Mindestlänge	12 Zeichen	Länge schlägt Komplexität (NIST SP 800-63B)
Maximallänge	128	DoS-Schutz beim Hashing
Komplexitätszwang	keiner	Erzeugt vorhersagbare Muster (<code>Password1!</code>)
Blocklist	Firmen-/Produktnamen (<code>lovedis</code> , <code>mara</code>), E-Mail-Lokalteil	Trivialraten
Breach-Check	HIBP Pwned Passwords, k-Anonymity	Nur die ersten 5 Zeichen des SHA-1-Hashes verlassen das System
Rotation	keine Zwangsrotation	Führt zu schwächeren Passwörtern; nur nach Vorfall

```
// src/lib/password-policy.ts
import { createHash } from "node:crypto";

/** HIBP k-Anonymity: es werden nur die ersten 5 Hex-Zeichen des SHA-1 übertragen. */
export async function isBreached(password: string): Promise<boolean> {
  const sha1 = createHash("sha1").update(password).digest("hex").toUpperCase();
  const res = await fetch(`https://api.pwnedpasswords.com/range/${sha1.slice(0, 5)}`, {
    headers: { "Add-Padding": "true" },
  });
  if (!res.ok) return false; // fail-open: Verfügbarkeit des Logins hat Vorrang
  return (await res.text()).split("\n").some((l) => l.startsWith(sha1.slice(5)));
}
```

Zu ergänzen in der Zod-Validierung von `src/app/actions/auth.ts` (Selbstregistrierung Partner/Startup) und im Reset-Flow.

10. Passwort- & Secret-Management

Hier sind **drei getrennte Themen** zu unterscheiden, die häufig vermischt werden:

1. **Endnutzer-Passwörter** in der Anwendung → §9.
2. **Anwendungs-Secrets** zur Laufzeit (DB-DSN, Auth-Secret, API-Keys) → §10.1–10.4.
3. **Team-Zugangsdaten** (Provider-Logins, SSH, Notion/Slack, Recovery-Codes) → §10.5.

10.1 Secret-Inventar (Soll-Zustand)

Name	Zweck	Klasse	Rotation	Ablage
<code>DATABASE_URL</code>	OLTP-DSN (<code>lovedis_app</code>)	K3	90 Tage	Secret-Manager
<code>DATABASE_URL_MIGRATE</code>	DDL-Rolle, nur CI	K3	90 Tage	CI-Secret
<code>DWH_DATABASE_URL</code>	DWH-Ziel	K3	90 Tage	Secret-Manager
<code>DWH_SOURCE_URL</code>	<code>lovedis_readonly</code> fürs ETL	K3	90 Tage	Secret-Manager
<code>AUTH_SECRET</code> / <code>NEXTAUTH_SECRET</code>	JWT-Signatur	K3	180 Tage, mit Overlap	Secret-Manager
<code>NEXTAUTH_URL</code>	öffentliche Basis-URL	K0	—	Env-Var (kein Secret)
<code>AUTH_TRUST_HOST</code>	Host-Vertrauen der Runtime	K0	—	Env-Var
<code>CRON_SECRET</code>	Bearer für <code>/api/cron/reminders</code>	K2	90 Tage	Secret-Manager
<code>FIELD_KEY_V1</code>	Feldverschlüsselung (§7.5)	K3	12 Monate, versioniert	KMS/Vault
<code>BLIND_INDEX_KEY</code>	HMAC für Suchindizes	K3	mit Reindex	KMS/Vault
<code>EMAIL_API_KEY</code>	EU-Mail-Provider	K2	180 Tage	Secret-Manager
<code>S3_ACCESS_KEY</code> / <code>S3_SECRET_KEY</code>	Objektspeicher (Uploads)	K2	90 Tage	Secret-Manager
<code>HIBP_PADDING</code>	—	K0	—	keine
<code>SENTRY_DSN</code> / <code>OTEL_*</code>	Telemetrie	K1	—	Env-Var
Deploy-SSH-Key / Registry-Token	Deployment	K3	180 Tage	CI-Secret, OIDC bevorzugt

Bewusst nicht mehr im Inventar: `CLOUDFLARE_API_TOKEN` (entfällt mit Szenario A). Der in `docs/deployment-plan-mara.md` §6 beschriebene Token-Blocker löst sich damit auf.

10.2 Regeln für Secrets

1. **Niemals im Repo** — auch nicht in `.env.example`, auch nicht kommentiert, auch nicht als „Test“-Wert. `.env` ist bereits gitignored; das bleibt so.
2. **Keine Secrets in `NEXT_PUBLIC_*`**. Alles mit diesem Präfix landet im Client-Bundle. Ein Lint-Check in CI, der `NEXT_PUBLIC_.*(KEY|SECRET|TOKEN|PASSWORD)` verbietet, verhindert den klassischen Unfall.
3. **Getrennte Secrets pro Umgebung**. Ein Staging-Secret darf in Produktion nichts öffnen.
4. **Least Privilege je Secret** — deshalb die vier DB-Rollen in §7.2.

5. **Kein Secret in Logs.** Redaction-Filter im Logger (`DATABASE_URL`, `authorization`, `cookie`, `password`, `token`) und Prisma-`query`-Log in Prod aus.
6. **Rotation ist ein geprobter Vorgang**, kein Notfall-Improvisation (§10.4).
7. **Vier-Augen-Prinzip** für den Zugriff auf `FIELD_KEY_V1` und `AUTH_SECRET`.

10.3 Secret-Store

Option	Bewertung
Infisical (self-hosted, DE) — Empfehlung	Vollständig EU/DE, gute DX, Versionierung, Audit-Log, Env-Trennung, CLI-Injection zur Deploy-Zeit, Kubernetes-/Docker-Integration
HashiCorp Vault (self-hosted)	Mächtigster Ansatz (dynamische DB-Credentials, Transit-Engine für Feldverschlüsselung), aber deutlich höhere Betriebslast
SOPS + age, im Git verschlüsselt	Sehr einfach, kein Server, gute Audit-Historie über Git — aber kein Widerruf einzelner Leser und kein Zugriffs-Audit
Provider-Secrets (<code>wrangler secret</code> , AWS Secrets Manager)	An den Provider gebunden; kein einheitlicher Ort über alle Systeme

Empfohlener Aufbau: **Infisical** als Single Source of Truth, Secrets werden **zur Deploy-Zeit** in den Container injiziert (nicht ins Image gebacken, nicht in der Shell-History). Perspektivisch (P2): Vault mit **dynamischen DB-Credentials** — kurzlebige, automatisch rotierende Postgres-Zugänge; dann existiert kein langlebiger `DATABASE_URL` mehr.

10.4 Rotationsverfahren

Wichtig für `AUTH_SECRET`: Eine harte Rotation invalidiert **alle** Sessions gleichzeitig (alle Nutzer werden ausgeloggt). Deshalb Overlap-Verfahren:

Secret	Verfahren
<code>AUTH_SECRET</code>	NextAuth mit Key-Rotation (alter Key nur zum <i>Verifizieren</i> , neuer zum <i>Signieren</i>), nach 8 h (max. Session-Lebensdauer) alten Key entfernen
<code>DATABASE_URL</code>	Zweiten DB-User anlegen → deployen → alten User löschen (kein Downtime-Fenster)
<code>FIELD_KEY_Vn</code>	Neue Version parallel; neue Schreibvorgänge mit <code>Vn+1</code> , Hintergrund-Reencryption der Altbestände, dann <code>Vn</code> entfernen — das Versionspräfix aus §7.5 macht das möglich
<code>BLIND_INDEX_KEY</code>	Nur mit vollständigem Reindex der Blind-Index-Spalten; deshalb selten rotieren
<code>CRON_SECRET</code>	Neuer Wert in Scheduler und App gleichzeitig; kurzes Fenster mit beiden akzeptierten Werten
Nach jedem Vorfall	Alle Secrets rotieren, unabhängig vom vermuteten Umfang

10.5 Team-Passwortmanagement

Aspekt	Vorgabe
Passwort-Manager	1Password Business (Server in der EU wählbar) oder Bitwarden (EU-Hosting bzw. self-hosted Vaultwarden). Pflicht für alle — kein Teilen über Slack, Notion, E-Mail oder Tabellen
Tresor-Struktur	Getrennte Vaults: <code>Infra-Prod</code> (nur Admins), <code>Infra-Staging</code> , <code>SaaS-Team</code> , <code>Notfall</code> (Break-Glass)
Master-Passwort	≥ 20 Zeichen bzw. Passphrase, nirgends digital notiert
Hardware-Keys	YubiKey (2 Stück pro Admin) für Passwort-Manager, GitHub, Provider-Konsolen, Google/Microsoft-Konto

Aspekt	Vorgabe
Break-Glass-Konto	Ein Notfall-Admin-Zugang, Passwort geteilt per Shamir-Split oder im physischen Safe, Nutzung löst Alarm aus
Recovery-Codes	Ausgedruckt im Safe, nicht im Passwort-Manager (sonst Single Point of Failure)
SSH	Nur <code>ed25519</code> -Keys mit Passphrase, besser: FIDO2-gebunden (<code>ed25519-sk</code>). Kein Passwort-Login, <code>PermitRootLogin no</code> , Zugang nur über WireGuard-Bastion
Provider-Konten	MFA erzwungen, keine geteilten Logins, jeder Admin ein eigenes Konto, Rechte nach Rolle
Onboarding	Rollenbasierte Vault-Freigabe, Hardware-Key-Ausgabe, Security-Briefing, dokumentierte Checkliste
Offboarding	innerhalb von 4 Stunden: Vault-Zugang entziehen, SSH-Keys entfernen, Provider-Konten deaktivieren, GitHub-Zugang entziehen, <code>sessionsValidFrom</code> setzen, alle geteilten Secrets rotieren, die die Person kannte

Das Offboarding ist der am häufigsten unterschätzte Punkt (Angreiferprofil A4). Ohne Rotation geteilter Secrets bleibt der Zugang faktisch bestehen, auch wenn das Konto gelöscht ist.

10.6 Endgeräte (Kurzfassung)

Vollverschlüsselung (FileVault/BitLocker) Pflicht, automatische Sperre nach 5 min, aktuelle OS-Patches, keine Produktionsdaten in lokalen Downloads/Exporten, keine Prod-DB-Zugriffe vom privaten Gerät. Wer `ADMIN` in Mara ist, arbeitet auf einem verwalteten Gerät.

11. Autorisierung, RBAC-Härtung & Audit

11.1 Rollen- und Sichtbarkeitsmatrix (Soll)

Abgeleitet aus `src/lib/auth-guards.ts` und `docs/mara-implementation-notes.md` (Entscheidung 7):

Datenbereich	ADMIN	MEMBER	BUSINESS_PARTNER	INVESTOR	STARTUP
Match-Matrix (<code>PartnerStartupMatch</code>)	✓ RW	✓ RW	✗	✗	✗
Interne Evaluations/Scores	✓ RW	✓ RW	✗	✗	✗
<code>Startup.screen*</code> (Erst-Einordnung)	✓ RW	✓ RW	✗	✗	✗
Longlist / Pushes / Credits-Admin	✓ RW	✓ RW	✗	✗	✗
Eigenes Partner-Verdikt	✓ R (Vorschau)	✓ R (Vorschau)	✓ RW nur eigenes	✗	✗
Fremde Partner-Verdikte	✓ R	✓ R	✗	✗	✗
Kuratierte Partner-Sichten (<code>/screening</code> , <code>/use-cases</code> , <code>/check-ins</code>)	✓ R	✓ R	✓	✗	✗
SSOT-Inhalte	✓ RW	✓ RW	✓ R (<code>audience</code>)	✗	✓ R (<code>audience</code>)
Marktplatz-Katalog	✓ RW	✓ RW	✗	✗	✓ R
Eigene Buchungen / Credits	✓ R	✓ R	✗	✗	✓ nur eigene
Öffentliche Startup-Storefront (<code>isPublished</code>)	✓	✓	✓	✓	✓
Nachrichten	nur eigene Konversationen	dito	dito	dito	dito
Nutzerverwaltung / Partner-Freigabe	✓	✗	✗	✗	✗
Audit-Log	✓ R	✗	✗	✗	✗

Zwei Regeln, die im Code erzwungen werden müssen:

- `BUSINESS_PARTNER` sieht **ausschließlich Zeilen mit eigener `partnerId`** — bei `PartnerStartupReview` , `StartupPush` , `CheckInReminder` , `Engagement` . Das ist der wahrscheinlichste IDOR-Pfad (Angreiferprofil A2) und genau der Fall, den die RLS-Policy in §7.4 zusätzlich absichert.
- `STARTUP` sieht **nie** `Evaluation` , `Score` , `screenRecommendation` oder Matrix-Zellen über sich selbst. Das ist geschäftlich zwingend — Startups dürfen ihr internes Ranking nicht kennen.

11.2 Objektbezogene Prüfung (IDOR-Schutz)

Rollenprüfung allein reicht nicht. Ein `STARTUP`-User mit gültiger Rolle darf nur *sein* Startup sehen. Muster: **immer im `where`-Clause filtern, nie nach dem Laden prüfen.**

```
// Falsch: lädt fremde Daten und filtert danach
const booking = await prisma.marketplaceBooking.findUnique({ where: { id } });
if (booking.startupId !== myStartupId) throw new Error("forbidden");

// Richtig: die Datenbank gibt fremde Zeilen nie zurück
const booking = await prisma.marketplaceBooking.findFirst({
  where: { id, startupId: myStartupId },
});
if (!booking) notFound(); // 404, nicht 403 – verrät nicht, dass die ID existiert
```

404 statt 403 ist bewusst: Ein 403 bestätigt die Existenz der Ressource und ermöglicht das Aufzählen fremder IDs.

11.3 Absicherung gegen vergessene Guards

Der strukturelle Schwachpunkt (F-12): Guards sind Konvention, nicht Zwang. Vier Gegenmaßnahmen:

1. **RLS in Postgres** (§7.4) — die Datenbank verweigert, was der Code vergisst.
2. **ESLint-Regel:** Jede exportierte `async function` in `src/app/actions/**` muss im Rumpf einen `require*`-Aufruf enthalten. Als lokale Regel oder `no-restricted-syntax`-Muster umsetzbar.
3. **Testabdeckung:** Für jede Action ein negativer Testfall pro nicht berechtigter Rolle (5 Rollen × N Actions). In `tests/` mit Vitest gut abbildbar.
4. **PR-Checkliste:** „Neue Server Action → Guard vorhanden? Objektbezogene Prüfung im `where`? Audit-Event geschrieben?“

11.4 Mandantentrennung Partner

`PartnerCompany` (Unternehmen) und `User` mit Rolle `BUSINESS_PARTNER` (Person) sind laut Schema-Kommentar bewusst getrennt — die Matrix funktioniert unabhängig davon, ob Partner-Accounts existieren. **Sicherheitsrelevante Lücke:** Es gibt derzeit **keine Verknüpfung** zwischen `User.company` (Freitext!) und `PartnerCompany`. Sobald Partner-Nutzer eigene unternehmensbezogene Daten sehen sollen, ist Freitext als Mandantengrenze untauglich.

Maßnahme: Explizite Relation einführen, bevor unternehmensbezogene Partner-Sichten gebaut werden:

```
model User {
  // ...
  partnerCompanyId String?
  partnerCompany PartnerCompany? @relation(fields: [partnerCompanyId], references: [id], onDelete: SetNull)
}

model PartnerCompany {
  // ...
  users User[]
}
```

Damit wird die Mandantengrenze ein Fremdschlüssel und kann in RLS-Policies und `where`-Clauses zuverlässig verwendet werden — statt eines String-Vergleichs auf einem frei editierbaren Feld.

11.5 Audit-Log (F-06)

Ohne Audit-Log ist ein Datenabfluss nicht nachweisbar — betrieblich wie rechtlich (DSGVO Art. 5 Abs. 2, Nachweispflicht). Das Log ist **append-only** (Trigger aus §7.3, Rolle `lovedis_audit` aus §7.2).

```

enum AuditAction {
    LOGIN_SUCCESS
    LOGIN_FAILED
    LOGOUT
    MFA_ENROLLED
    MFA_FAILED
    PASSWORD_CHANGED
    PASSWORD_RESET_REQUESTED
    ROLE_CHANGED
    PARTNER_APPROVED
    USER_DEACTIVATED
    RECORD_CREATED
    RECORD_UPDATED
    RECORD_DELETED
    SENSITIVE_VIEWED // K3-Zugriff: Matrix, Evaluations, fremde Verdikte
    EXPORT_PERFORMED // CSV/XLSX/PDF
    CREDIT_BOOKED
    PERMISSION_DENIED
}

model AuditEvent {
    id          String          @id @default(cuid())
    action       AuditAction
    actorId      String?        // null bei fehlgeschlagenem Login (Konto unbekannt)
    actorRole    UserRole?
    entityType   String?        // "PartnerStartupMatch", "Evaluation", ...
    entityId     String?
    // Nur Feldnamen + Vorher/Nachher-Hashes bei K3-Feldern, KEINE Klartextwerte –
    // sonst wird das Audit-Log selbst zum Datenleck.
    changes      Json?
    ipHash       String?        // HMAC, kein Klartext (§17.2)
    userAgent    String?
    requestId    String?        // Korrelation mit Applikationslogs
    createdAt    DateTime       @default(now())

    @@index([actorId, createdAt])
    @@index([entityType, entityId])
    @@index([action, createdAt])
    @@index([createdAt])
}

```

Was zwingend geloggt wird:

Ereignis	Warum
Jeder Zugriff auf die Match-Matrix	kritischster Datensatz (§1.1)
Jeder Blick auf Evaluation / Score eines Startups	K3
Jedes Partner-Verdikt (Anlage/Änderung)	Nichtabstreitbarkeit
Jede CreditTransaction	Geldwert
Jeder Export (CSV/XLSX/PDF) inkl. Umfang und Filter	Exfiltration durch Berechtigte (A4)
Rollenwechsel, Partner-Freigaben, Deaktivierungen	Privilegienverwaltung
Jeder abgewiesene Zugriff (PERMISSION_DENIED)	Angriffserkennung

Aufbewahrung: 12 Monate in der DB, danach Export in unveränderlichen Objektspeicher. **SENSITIVE_VIEWED** erzeugt Volumen — ein Read-Audit auf der Matrix wird der größte Anteil sein. Deshalb: monatliche Partitionierung von **Audit-Event** einplanen.

11.6 Export-Kontrolle (F-20)

Heute laufen Exporte über `jspdf` / `html2canvas` / `xlsx` **im Browser**. Konsequenz: Sie sind unsichtbar für den Server — nicht auditierbar, nicht limitierbar, nicht wasserzeichenfähig. Für Angreiferprofil A4 (Ex-Mitarbeiter) ist das der bequemste Abflusskanal.

Maßnahme	Umsetzung
Exporte serverseitig erzeugen	Server Action liefert den Datenstrom; Client rendert nicht selbst
Jeder Export → <code>EXPORT_PERFORMED</code>	Actor, Umfang, Filter, Zeilenzahl
Mengenbegrenzung	z. B. max. 500 Zeilen/Export, max. 10 Exporte/Tag/User
Wasserzeichen	Name, E-Mail und Zeitstempel des Exportierenden in PDF/XLSX
Matrix-Vollexport	Nur <code>ADMIN</code> , mit Vier-Augen-Bestätigung
Alarm	> 3 Exporte in 10 Minuten → Meldung ins Monitoring (§16.4)

12. Anwendungssicherheit

12.1 OWASP Top 10 gegen den konkreten Code

Risiko	Aktueller Stand	Maßnahme
A01 Broken Access Control	Rollen-Gates vorhanden, aber nur applikativ (F-12)	RLS §7.4, IDOR-Muster §11.2, Guard-Lint §11.3
A02 Cryptographic Failures	bcryptjs (F-09), Klartextfelder (F-11)	Argon2id §9.2, Feldverschlüsselung §7.5, TLS §13.4
A03 Injection	Prisma parametrisiert; Risiko nur bei <code>\$queryRaw</code> / <code>\$executeRaw</code>	Nur <code>Prisma.sql</code> -Templates, nie String-Konkatenation; CI-Grep auf <code>\$queryRawUnsafe</code>
A04 Insecure Design	Score/Credit-Manipulation	DB-Constraints §7.3, Append-only-Ledger
A05 Security Misconfiguration	keine Header/CSP (F-05), <code>/odie</code> public (F-18)	§12.2, Public-Path-Review
A06 Vulnerable Components	<code>xlsx@0.18.5</code> (F-07), NextAuth Beta (F-13)	§18.3, §18.4
A07 Auth Failures	kein Rate-Limit/MFA/Reset (F-02/03/04)	§9
A08 Integrity Failures	kein SBOM, keine Lock-Verifikation	§18.2
A09 Logging Failures	kein Audit-Log (F-06)	§11.5, §16
A10 SSRF	<code>Attachment.url</code> / <code>MediaAsset.url</code> frei (F-10)	§12.6

12.2 Security-Header & CSP (F-05)

```
// next.config.ts
import type { NextConfig } from "next";

const isProd = process.env.NODE_ENV === "production";

// Next.js benötigt für Inline-Bootstrap-Skripte entweder 'unsafe-inline' oder Nonces.
// Nonces sind der sichere Weg – sie werden in der Middleware pro Request gesetzt.
const csp = [
  "default-src 'self'",
  "script-src 'self' 'nonce-__NONCE__' 'strict-dynamic'",
  // Tailwind v4 und CSS-in-JS benötigen Inline-Styles.
  "style-src 'self' 'unsafe-inline'",
  "img-src 'self' data: blob: https:",
  "font-src 'self' data:",
  // Nur die eigene API + HIBP-Range-Endpunkt (§9.7).
  "connect-src 'self' https://api.pwnedpasswords.com",
  "frame-ancestors 'none'",
  "form-action 'self'",
  "base-uri 'self'",
  "object-src 'none'",
  isProd ? "upgrade-insecure-requests" : "",
].filter(Boolean).join("; ");

const nextConfig: NextConfig = {
  poweredByHeader: false, // kein "X-Powered-By: Next.js"
  serverExternalPackages: ["@prisma/client", "@prisma/adapter-pg", "pg"],
  async headers() {
    return [{
      source: "/:path*",
      headers: [
        { key: "Content-Security-Policy", value: csp },
        { key: "Strict-Transport-Security", value: "max-age=63072000; includeSubDomains; preload" },
        { key: "X-Content-Type-Options", value: "nosniff" },
        { key: "X-Frame-Options", value: "DENY" },
        { key: "Referrer-Policy", value: "strict-origin-when-cross-origin" },
        { key: "Permissions-Policy", value: "camera=(), microphone=(), geolocation=(), payment=()" },
        { key: "Cross-Origin-Opener-Policy", value: "same-origin" },
        { key: "Cross-Origin-Resource-Policy", value: "same-origin" },
        { key: "X-DNS-Prefetch-Control", value: "off" },
      ],
    }],
  };
};

export default nextConfig;
```

Wichtig: Mit Szenario A entfallen `@prisma/adapter-neon`, `@neondatabase/serverless` aus `serverExternalPackages` und der komplette `outputFileTracingIncludes`-Workaround für `pg-cloudflare`.

Vorgehen für die CSP: Zuerst `Content-Security-Policy-Report-Only` mit `report-uri` / `report-to` ausrollen, 2 Wochen Verstöße sammeln (insbesondere durch `html2canvas`, `recharts` und Tailwind), dann scharf stellen. Eine direkt scharf gestellte CSP bricht in einer React-App zuverlässig irgendetwas.

HSTS-Vorsicht: `includeSubDomains; preload` gilt für die **gesamte Domain** `lovedis.de`. Vor der Aktivierung sicherstellen, dass jede Subdomain HTTPS spricht — sonst wird die Live-Homepage mitbetroffen. Erst auf `app.lovedis.de` ohne `includeSubDomains` testen.

12.3 Server Actions

Server Actions sind hier die **einzige Schreib-API** — das ist architektonisch gut, verlagert aber die gesamte Verantwortung in diese Funktionen:

Regel	Grund
Jede Action beginnt mit <code>require*()</code>	Autorisierung (§11.3)
Jede Action validiert mit <code>Zod, .strict()</code>	Verhindert Mass Assignment über unerwartete Felder
Ausgangswerte explizit in <code>data: {}</code> aufführen, kein Spread von Client-Objekten	Ein <code>...input</code> kann <code>role</code> , <code>approvedAt</code> oder <code>balance</code> setzen
Nie <code>id</code> oder <code>role</code> aus dem Client-Payload übernehmen	Privilege Escalation
Nach Schreibvorgang <code>revalidatePath</code> + Audit-Event	Konsistenz + Nachweis
Fehler generisch nach außen, Details nur ins Log	Keine Stacktraces/DB-Fehler an den Client
Origin-Check aktiv lassen	Next.js schützt Server Actions per Origin-Header gegen CSRF; keine eigenen Bypässe bauen

```
// Muster für eine Action mit K3-Bezug
const schema = z.object({
  startupId: z.string().cuid(),
  verdict: z.enum(["PENDING", "CONTINUE", "PASS"]),
  note: z.string().max(4000).optional(),
}).strict();

export async function submitPartnerVerdict(input: unknown) {
  const session = await requirePartner(); // 1. Rolle
  if (!(await isPartnerApproved(session.user.id))) return { error: "not_approved" }; // 2. Freigabe
  const data = schema.parse(input); // 3. Validierung
  const review = await prisma.partnerStartupReview.upsert({
    // 4. partnerId kommt aus der SESSION, nie aus dem Payload
    where: { partnerId_startupId_challengeId: {
      partnerId: session.user.id, startupId: data.startupId, challengeId: null } },
    create: { partnerId: session.user.id, startupId: data.startupId,
      verdict: data.verdict, note: data.note && encryptField(data.note) },
    update: { verdict: data.verdict, note: data.note && encryptField(data.note) },
  });
  await writeAudit({ action: "RECORD_UPDATED", session, // 5. Audit
    entityType: "PartnerStartupReview", entityId: review.id });
  revalidatePath("/screening");
}
```

12.4 XSS

- React escapet standardmäßig. **Kritischer Punkt:** `ContentPage.body` ist **Markdown** (`schema.prisma:761`) und wird von Team-Nutzern über `/hub-admin` gepflegt. Wird das mit `dangerouslySetInnerHTML` oder einem Markdown-Renderer ohne Sanitizing dargestellt, ist es ein Stored-XSS-Vektor gegen Partner und Startups.
- **Maßnahme:** Markdown mit `rehype-sanitize` (Allowlist-Schema) rendern, **niemals** rohes HTML durchlassen. Zusätzlich CSP als zweite Verteidigungslinie.
- CI-Grep auf `dangerouslySetInnerHTML` mit Pflicht-Begründung im PR.
- `MediaAsset.url` / `Attachment.url` / `Startup.website` / `MentorProfile.website`: Schema auf `https:` beschränken — `javascript:` und `data:` URLs blocken (§12.6).

12.5 Eingabevalidierung

Feld	Prüfung
Alle URL-Felder (<code>website</code> , <code>url</code> , <code>logoUrl</code> , <code>photoUrl</code>)	<code>z.string().url()</code> plus Protokoll-Allowlist <code>https:</code>
Freitexte (<code>note</code> , <code>body</code> , <code>message</code> , <code>pitch</code>)	<code>.max()</code> setzen — sonst DoS über Megabyte-Strings
<code>Score.value</code>	<code>z.number().int().min(0).max(5)</code> (+ DB-Constraint §7.3)
Credit-Beträge	<code>z.number().int()</code> , plausible Obergrenze, serverseitige Saldoprüfung
E-Mail	<code>z.email()</code> + Lowercase-Normalisierung (in <code>auth.ts</code> bereits korrekt)
Array-Felder (<code>focusTags</code> , <code>lookingFor</code> , <code>expertise</code>)	Längen- und Elementlimits
Datei-Uploads	§12.6
Paginierung (<code>take</code> / <code>skip</code>)	Obergrenze erzwingen, sonst <code>take: 999999</code> -DoS

12.6 Dateien & Uploads (F-10)

Heute werden nur **URLs** gespeichert (`Attachment.url` , `MediaAsset.url`). Das bedeutet: Pitchdecks liegen irgendwo, ohne dass die Plattform Zugriff kontrolliert. Wer die URL kennt, hat die Datei — ein klassischer „Security by Obscurity“-Fall bei K2/K3-Dokumenten.

Zielbild:

Aspekt	Vorgabe
Speicherort	Privater S3-Bucket (Hetzner Object Storage, DE), kein Public Read
Zugriff	Presigned URLs, 5 Minuten gültig , ausgestellt erst nach Rollenprüfung
Upload	Presigned PUT, <code>Content-Length</code> -Limit 25 MB , MIME-Allowlist (PDF, PPTX, DOCX, XLSX, PNG, JPG)
Typprüfung	Magic Bytes serverseitig prüfen, nicht die Dateieindung und nicht den Client- <code>Content-Type</code>
Virenskan	ClamAV-Scan asynchron nach Upload; Datei bleibt bis Freigabe in Quarantäne
Ausliefern	<code>Content-Disposition: attachment</code> + <code>X-Content-Type-Options: nosniff</code> ; niemals HTML/SVG inline rendern
Isolation	Auf separater Domain (<code>files.lovedis.de</code>) ausliefern, damit ein Datei-XSS nicht die App-Origin trifft
Metadaten	Uploader, Zeitstempel, Prüfsumme (SHA-256) und Größe persistieren
SSRF	Externe URLs nie serverseitig abrufen; wenn doch nötig (Link-Preview): DNS-Auflösung prüfen und private IP-Bereiche (<code>10/8</code> , <code>172.16/12</code> , <code>192.168/16</code> , <code>169.254/16</code> , <code>127/8</code> , <code>::1</code> , <code>fc00::/7</code>) blocken

Schema-Ergänzung:

```
model Attachment {
  // ...
  url          String? // nur für type = LINK (externe Referenz)
  storageKey   String? // privater Objekt-Key für type = DOCUMENT/DECK
  mimeType     String?
  sizeBytes    Int?
  sha256       String?
  scanStatus   String @default("PENDING") // PENDING | CLEAN | INFECTED
  uploadedById String?
}
```


13. Edge-, Netz- & Transportsicherheit

13.1 WAF

Anbieter	Bewertung
Myra Security (DE) — Empfehlung für Szenario A	BSI-C5-Testat, §8a-BSIG-qualifiziert, deutsche Vertragskette und deutscher Support. Im Partner-Audit das stärkste Argument. Preislich über Cloudflare
Link11 (DE)	Ebenfalls deutsch, DDoS-Fokus, gute Alternative
Cloudflare WAF	Technisch führend, günstigste Option — aber US-Vertragskette (§17.5 TIA erforderlich)
Baqend/Fastly/Akamai	Für diesen Anwendungsfall überdimensioniert

Regelwerk (unabhängig vom Anbieter):

- OWASP Core Rule Set aktiv, zunächst im Log-Modus, dann blockend.
- Rate-Limits:** `/api/auth/*` 10/5 min/IP · `/login` 20/5 min/IP · globaler Fallback 600/min/IP · Export-Endpunkte 10/h/Session.
- Geo-Regeln:** Admin-Pfade (`/hub-admin` , `/matrix` , `/credits`) auf DE/EU begrenzen, sofern das Team nicht außerhalb arbeitet.
- Bot-Management gegen Scraping der öffentlichen Startup-Storefront.
- Origin-Schutz:** Die App-Nodes akzeptieren **nur** Verbindungen von den WAF-IPs (Firewall) plus ein Shared Secret im Header. Ohne das kann jeder die WAF umgehen, indem er die Origin-IP direkt anspricht — der häufigste WAF-Bypass in der Praxis.

13.2 Firewall & Netzzugang

Regel	Wert
Eingehend App-Nodes	443 nur von WAF-IPs; 22 nur von der WireGuard-Bastion
Eingehend DB	5432 nur aus dem App-Subnetz + Bastion
Ausgehend App	Allowlist: DB, Objektspeicher, Mail-Provider, <code>api.pwnedpasswords.com</code> , Telemetrie. Kein offenes Egress — begrenzt Datenabfluss und C2-Kanäle nach einer Kompromittierung
Management	Kein Provider-Konsolen-Zugriff ohne MFA
SSH	Key-only, <code>PermitRootLogin no</code> , <code>PasswordAuthentication no</code> , fail2ban

13.3 DDoS & Missbrauch

- Volumetrisch: WAF-Anbieter (Anycast-Scrubbing).
- Applikativ: Rate-Limits (§13.1), `statement_timeout` (§7.1), Upload-Limits (§12.6), Paginierungsgrenzen (§12.5).
- Kostenschutz: Budget-Alarme beim Hoster; in Szenario A begrenzen Fixpreise das Risiko einer „Denial of Wallet“ ohnehin stark — ein Vorteil gegenüber verbrauchsbasierten Serverless-Modellen.

13.4 TLS

Aspekt	Vorgabe
Protokolle	nur TLS 1.3 (1.2 mit AEAD-Ciphern als Übergang), kein TLS ≤ 1.1
Zertifikate	Let's Encrypt via Caddy, automatische Erneuerung, Monitoring auf Restlaufzeit < 21 Tage
HSTS	<code>max-age=63072000; includeSubDomains; preload</code> — Vorsicht §12.2
CAA-Record	<code>lovedis.de. CAA 0 issue "letsencrypt.org"</code> — verhindert Fehl-Ausstellung durch andere CAs
DB-Verbindung	<code>sslmode=verify-full</code> mit <code>sslrootcert</code> — nicht <code>require</code> (das prüft den Hostnamen nicht)
Interner Verkehr	Auch im privaten Netz TLS; „internes Netz = vertrauenswürdig“ ist keine tragfähige Annahme

13.5 DNS & E-Mail-Sicherheit

Record	Wert	Zweck
SPF	<code>v=spf1 include:<eu-provider> -all</code>	Nur autorisierte Sender; <code>-all</code> , nicht <code>~all</code>
DKIM	2048-bit, Provider-Schlüssel	Signatur
DMARC	<code>v=DMARC1; p=reject; rua=mailto:dmARC@lovedis.de; pct=100</code>	Schutz vor Spoofing der Marke — relevant, weil Mara Erinnerungs-Mails an Partner sendet
MTA-STS + TLS-RPT	<code>enforce</code>	Transportverschlüsselung erzwingen
CAA	<code>letsencrypt.org</code>	Zertifikatskontrolle
DNSSEC	aktiv	Schutz gegen DNS-Spoofing
Registrar	Transfer-Lock + MFA	Domain-Hijacking ist der Totalverlust

Weg zu `p=reject`: mit `p=none` starten, DMARC-Reports 2–4 Wochen auswerten, dann `quarantine`, dann `reject`. Direkt `reject` kann legitime Mails (z. B. aus Notion oder Slack-Integrationen) verwerfen.

14. Mara-Integration: Datenflüsse & Fremdsysteme

14.1 Was „Mara-Integration“ konkret bedeutet

Mara ist kein externes System, das angebunden wird, sondern die **Anwendungsschicht auf demselben Datenmodell**: Screening (`PartnerStartupReview` , `Startup.screen*`), Partner-SSOT (`RoadmapItem` , `ContentPage` , `Media-Asset` , `KnowledgeResource`), Marktplatz (`Program` , `MentorProfile` , `SupportOffering` , `MarketplaceBooking`), Credits (`CreditAccount` , `CreditTransaction`), Match-Matrix (`PartnerCompany` , `PartnerStartupMatch`).

Sicherheitsrelevante Konsequenz: Es gibt **keine** technische Trennung zwischen Homepage-Public, Partner-Sicht, Startup-Sicht und internen Team-Daten außer der Rollenlogik. Die Trennung ist **logisch, nicht physisch**. Genau deshalb sind RLS (§7.4), Audit (§11.5) und die Mandanten-Fremdschlüssel (§11.4) keine Optionen, sondern das Fundament.

14.2 Zonierung der Oberflächen

Zone	Routen	Datenklasse	Gate
Public	Homepage, veröffentlichte Storefront	K0	keiner
Startup-Self-Service	<code>/venture/*</code> , <code>/venture/marketplace/*</code>	K2 (eigene)	<code>requireStartup</code> / <code>requireVentureView</code>
Partner-kuratiert	<code>/screening</code> , <code>/use-cases</code> , <code>/check-ins</code> , <code>/partner-hub</code>	K2/K3 (eigene)	<code>requirePartner</code> / <code>requirePartnerView</code> + <code>approvedAt</code>
Investor	Marktplatz-Entdeckung, <code>IntroRequest</code>	K2	<code>requireMarketplace</code>
Team intern	<code>/longlist</code> , <code>/pushes</code> , <code>/credits</code> , <code>/hub-admin</code> , <code>/engagements</code> , <code>/marketplace</code> , Matrix	K3	<code>requireTeam</code> / <code>requireScoutModule</code> + MFA (§9.3)

14.3 Homepage ↔ Plattform

Der bestehende Grundsatz aus `docs/deployment-plan-mara.md` bleibt gültig und wird sicherheitstechnisch verstärkt:

Aspekt	Vorgabe
Trennung	Homepage und Plattform bleiben getrennte Deployments auf getrennten Hosts (<code>lovedis.de</code> vs. <code>app.lovedis.de</code>)
Cookie-Isolation	Session-Cookie mit <code>__Host-</code> -Präfix → kein <code>Domain=.lovedis.de</code> . Damit kann die Homepage (oder eine kompromittierte Subdomain) niemals auf das Plattform-Cookie zugreifen
Kein geteiltes Secret	Homepage und Plattform teilen weder DB noch Secrets
Übergang	Nur ein Link „Login“ von der Homepage zur Plattform, kein SSO-Durchgriff, kein geteilter State
DNS	Nur ein zusätzlicher Record für <code>app.</code> ; Apex und <code>www</code> bleiben unberührt

Die Cookie-Isolation ist der wichtigste Punkt: Ohne `__Host-`-Präfix und mit domainweitem Cookie würde eine XSS-Lücke auf der (weniger geschützten) Marketing-Homepage Plattform-Sessions kompromittieren.

14.4 Externe Systeme

System	Status	Datenfluss	Sicherheitsanforderung
E-Mail	<code>consoleEmailAdapter</code> , kein Provider	Erinnerungen an Partner (Name, E-Mail, Startup-Kontext)	EU-Provider (Brevo/Mailjet/Scaleway TEM), AVV, minimaler Inhalt: keine K3-Daten in der Mail , nur Link in die Plattform
Cron/Scheduler	Endpoint fertig, Trigger fehlt	POST <code>/api/cron/reminders</code>	<code>CRON_SECRET</code> pflichtig (bereits fail-closed in Prod), Aufrufer-IP-Allowlist, Timing-safe Vergleich, eigenes Rate-Limit
Notion	manuelle Migration geplant	Inhalte → <code>ContentPage</code> / <code>RoadmapItem</code>	Nur Einbahn Notion → Plattform, Token mit minimalem Scope, keine K3-Daten in Notion
Attio (CRM)	nur dokumentiert, kein Sync	perspektivisch Startup-Stammdaten	Nie Match-Matrix oder Verdikte synchronisieren; nur K0/K1-Felder; separater AVV
Slack	für Alerting relevant	Betriebsmeldungen	Keine K2/K3-Daten in Slack-Nachrichten — nur „Ereignis X, Details in der Plattform“
HIBP	neu (§9.7)	5 Zeichen eines SHA-1-Präfixes	k-Anonymity, kein Personenbezug übertragen

Grundregel für jede künftige Integration: Ein neues Fremdsystem erhält **niemals** Zugriff auf `PartnerStartupMatch`, `PartnerStartupReview`, `Evaluation`, `Score` oder `Message`. Diese fünf Modelle verlassen die Plattform nicht — außer in pseudonymisierter Form ins DWH (§15.6).

14.5 Wenn später eine echte API entsteht

Sobald Partner programmatisch zugreifen sollen (heute nicht der Fall — es gibt nur `/api/auth`, `/api/health`, `/api/cron`, `/api/session-clear`):

- Eigener Pfad `/api/v1/*`, versioniert, **getrennt** von den internen Server Actions.
- Authentifizierung über kurzlebige Tokens (OAuth2 Client Credentials), nicht über Session-Cookies; Scopes je Datenbereich.
- Mandanten-Scope **im Token**, plus RLS als Netz darunter.
- Rate-Limit pro Client, Audit jedes Aufrufs, dokumentierte Deprecation-Politik.
- Keine „internen“ Felder in Response-DTOs — explizite Mapping-Layer statt Prisma-Objekte direkt zu serialisieren (sonst leakt das nächste Schema-Feld automatisch mit).

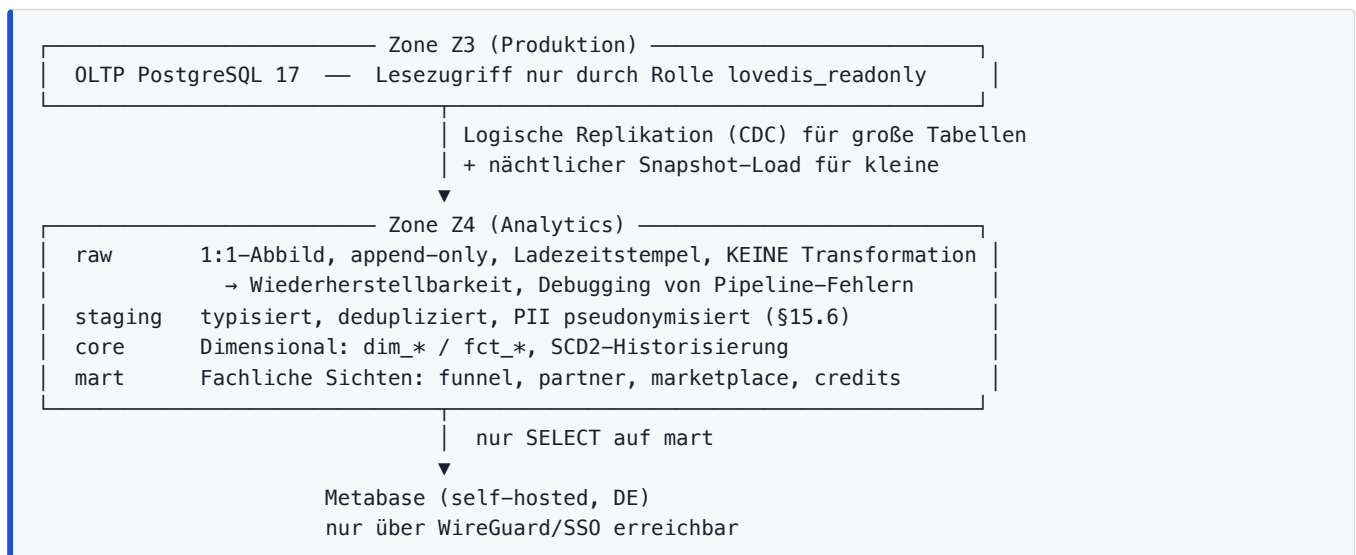
15. Data Warehousing

15.1 Warum überhaupt ein DWH

Heute laufen Auswertungen (Reports, Radar, Vergleich, Match-Matrix) **direkt gegen die Produktionsdatenbank**, teilweise mit clientseitiger Aggregation. Vier Probleme:

1. **Performance-/Verfügbarkeitsrisiko:** Eine teure Analyse-Query kann den OLTP-Betrieb beeinträchtigen.
2. **Keine Historie:** Das Schema speichert fast überall nur den **aktuellen Stand**. `Startup.pipelineStage` wird überschrieben (F-17) — „Wie lange lag ein Startup in `SCREENING` ?“ und „Wie hat sich die Conversion über die Batches entwickelt?“ sind heute **nicht beantwortbar**. Das ist die eigentliche fachliche Lücke.
3. **Sicherheitsrisiko:** Analytische Zugriffe brauchen breite Leserechte auf K3-Daten. Jeder BI-Zugang auf der Produktions-DB ist ein potenzieller Vollzugriff.
4. **Keine Governance:** Kennzahlen werden ad hoc in Komponenten berechnet; es gibt keine verbindliche Definition von „aktives Startup“ oder „Conversion“.

15.2 Zielarchitektur (Medaillen-Schichten)



Technologiewahl: PostgreSQL als DWH — kein ClickHouse, kein Snowflake, kein BigQuery. Begründung: Die Datenmengen liegen bei tausenden, nicht Milliarden Zeilen. Postgres mit materialisierten Sichten und passenden Indizes ist für diese Größenordnung um Größenordnungen einfacher zu betreiben, bleibt in der EU, kostet ~25 €/Monat und benötigt kein neues Know-how. **Erst** wenn Event-Volumen (z. B. Audit-Reads, Klickströme) die 100-Millionen-Zeilen-Marke überschreitet, ist ClickHouse (self-hosted in DE) der nächste Schritt.

Transformationen mit **dbt-core** (Versionierung im Repo, Tests, Dokumentation, Lineage), orchestriert per systemd-Timer oder GitHub-Actions-Runner im privaten Netz.

15.3 Ladeverfahren

Quelle	Verfahren	Frequenz
Große/häufig geänderte Tabellen (<code>Startup</code> , <code>Evaluation</code> , <code>Score</code> , <code>MarketplaceBooking</code> , <code>CreditTransaction</code> , <code>AuditEvent</code>)	Logische Replikation (Publication/Subscription) → <code>raw</code>	kontinuierlich

Quelle	Verfahren	Frequenz
Kleine Stammdaten (<code>PartnerCompany</code> , <code>Program</code> , <code>MentorProfile</code> , <code>SupportOffering</code> , Enums)	Snapshot-Load	nächtlich 02:00
<code>PartnerStartupMatch</code>	Snapshot mit Historisierung (SCD2)	nächtlich
<code>Message</code>	wird nicht geladen (nur aggregierte Zähler)	—
<code>User</code>	nur pseudonymisiert (Hash, Rolle, Firma)	nächtlich

Prinzipien: Die ETL-Rolle ist `lovedis_readonly` mit `default_transaction_read_only = on` — ein fehlerhaftes ETL kann Produktionsdaten **nicht** beschädigen. Läufe sind idempotent und wiederholbar, jeder Lauf wird protokolliert (Start, Dauer, Zeilen, Fehler), und ein fehlender Lauf löst nach 26 Stunden einen Alarm aus (Silent Failure ist der Normalfall bei ETL).

15.4 Dimensionales Modell

Dimensionen

Tabelle	Typ	Inhalt	Anmerkung
<code>dim_date</code>	statisch	Tag, Woche, Monat, Quartal, Geschäftsjahr	Standard-Kalenderdimension
<code>dim_startup</code>	SCD2	Name, Branche, Stage, Land, Gründungsjahr, <code>pipelineStage</code> , Radar-Einordnung	Löst F-17: Stage-Verlauf wird historisiert
<code>dim_partner_company</code>	SCD1	Name, Slug, Sortierung	
<code>dim_user</code>	SCD1, pseudonymisiert	<code>user_key</code> (HMAC), Rolle, Firma, Aktiv-Flag — kein Name, keine E-Mail	§15.6
<code>dim_challenge</code>	SCD1	Titel, Status, Deadline, Tags	
<code>dim_offering</code>	SCD1	Programme/Mentoren/Support vereinheitlicht, Typ, Kategorie, Credit-Preis	
<code>dim_campaign</code>	SCD1	Batch/Longlist (<code>ScoutingCampaign</code>)	

Fakten

Tabelle	Granularität	Kennzahlen	Quelle
<code>fct_evaluation</code>	eine Bewertung	<code>overall_score</code> , Empfehlung, Dimensionswerte (pivotiert)	<code>Evaluation</code> + <code>Score</code>
<code>fct_pipeline_transition</code>	ein Stufenwechsel	<code>days_in_previous_stage</code>	neu , aus <code>StartupStageEvent</code> (§15.5)
<code>fct_partner_review</code>	ein Verdikt	Verdikt, Zeit bis Verdikt	<code>PartnerStartupReview</code>
<code>fct_push</code>	ein Push	Zeit bis erste Reaktion	<code>StartupPush</code>
<code>fct_booking</code>	eine Buchung	<code>credit_cost</code> , <code>fix_credit_cost</code> , Statusdauern, Storno-Flag	<code>MarketplaceBooking</code>
<code>fct_credit_tx</code>	eine Buchung	<code>amount</code> , <code>bucket</code> , Typ	<code>CreditTransaction</code>
<code>fct_match_cell</code>	Paarung Partner × Startup (SCD2)	Relevanz beidseitig, Kontaktstatus, Use-Case-Typen	<code>PartnerStartupMatch</code>

Tabelle	Granularität	Kennzahlen	Quelle
<code>fct_engagement</code>	ein Engagement	Dauer, Meilenstein-Erfüllungsgrad	<code>Engagement</code>
<code>fct_message_activity</code>	Tag × Konversation	nur Anzahl , nie Inhalt	<code>Message</code>
<code>fct_reminder</code>	eine Erinnerung	Pünktlichkeit, Erledigungsquote	<code>CheckInReminder</code>
<code>fct_security_event</code>	ein Audit-Event	Login-Fehlerquoten, Exportvolumen, <code>PERMISSION_DENIED</code>	<code>AuditEvent</code>

`fct_security_event` ist bewusst enthalten: Sicherheitsmonitoring profitiert von denselben Auswertungswerkzeugen (§16.4).

15.5 Voraussetzung im OLTP-Schema: Stage-Historie

Ohne dieses Modell ist Funnel-Analytik unmöglich (F-17). Es dient gleichzeitig als fachlicher Audit-Trail:

```

model StartupStageEvent {
  id          String          @id @default(cuid())
  startupId   String
  startup     Startup         @relation(fields: [startupId], references: [id], onDelete: Cascade)
  fromStage   PipelineStage?
  toStage     PipelineStage
  changedById String?
  changedBy   User?          @relation("StageChanger", fields: [changedById], references: [id], onDelete: SetNull)
  note        String?
  createdAt   DateTime        @default(now())

  @@index([startupId, createdAt])
  @@index([toStage, createdAt])
}

```

Geschrieben wird in derselben Transaktion, die `Startup.pipelineStage` ändert — sonst driften Zustand und Historie auseinander.

15.6 Governance, Pseudonymisierung & Zugriff im DWH

Grundsatz: Das DWH enthält keine Klardaten von Personen und keine vertraulichen Freitexte.

Regel	Umsetzung
Keine Klartext-PII	<code>dim_user.user_key = HMAC-SHA256(user_id, DWH_PSEUDONYM_KEY)</code> ; Name/E-Mail werden nicht geladen
Keine Nachrichteninhalte	<code>Message.body</code> wird nie extrahiert; nur Zähler in <code>fct_message_activity</code>
Keine vertraulichen Freitexte	<code>PartnerStartupReview.note</code> , <code>Evaluation.notes</code> , <code>screenSummary</code> bleiben im OLTP; im DWH nur Flag „Notiz vorhanden“ und Länge
Getrennter Schlüssel	<code>DWH_PSEUDONYM_KEY</code> ≠ <code>BLIND_INDEX_KEY</code> , liegt nur im ETL-Kontext — die Re-Identifikation ist damit im DWH nicht möglich
Kontaktdaten	<code>Contact</code> , <code>MarketplaceBooking.contactEmail</code> werden nicht geladen
Netzzugang	DWH und Metabase ohne öffentliche IP, erreichbar nur über WireGuard
Zugriffsrechte	BI-User hat SELECT ausschließlich auf <code>mart</code> — nicht auf <code>raw</code> oder <code>staging</code>
Row-/Column-Level in BI	Metabase-Sandboxing, falls Partner je eigene Dashboards erhalten

Regel	Umsetzung
Aufbewahrung	<code>raw</code> 90 Tage, <code>staging</code> 180 Tage, <code>core</code> / <code>mart</code> gemäß §3.2
Anonymisierungs-Schwelle	Kennzahlen mit weniger als 5 zugrundeliegenden Entitäten werden unterdrückt (keine Rückrechnung auf Einzelfälle)
Backup	DWH-Backups separat, gleiche Verschlüsselungsregeln wie §8

Maskierungsskript für Staging (behebt das Risiko, dass Prod-Daten in Testumgebungen landen — §6.1 Regel 1): E-Mails → `user-<n>@example.invalid`, Namen → generierte Werte, `passwordHash` → Hash eines Wegwerf-Passworts, `Message.body` → Lorem, `Contact.*` → Fake, `PartnerStartupMatch` → vollständig geleert oder mit Zufallswerten befüllt.

15.7 Kennzahlen (verbindliche Definitionen)

Zentrale Definitionen verhindern, dass zwei Auswertungen unterschiedliche „Wahrheiten“ liefern.

Kennzahl	Definition	Mart
Funnel-Conversion	Anteil Startups, die von Stufe X nach Y wechseln, je Kampagne	<code>mart_funnel</code>
Time-in-Stage	Median Tage zwischen zwei <code>StartupStageEvent</code> s	<code>mart_funnel</code>
Screening-Durchsatz	Startups mit <code>screenedAt</code> je Woche und Screener	<code>mart_screening</code>
Partner-Responsivität	Median Tage von <code>StartupPush.createdAt</code> bis erstem <code>PartnerStartupReview</code>	<code>mart_partner</code>
Verdikt-Quote	Anteil <code>CONTINUE</code> / (<code>CONTINUE</code> + <code>PASS</code>) je Partner	<code>mart_partner</code>
Matrix-Abdeckung	Anteil befüllter Zellen (beide Relevanzen gesetzt) je Partner	<code>mart_matrix</code>
Pilot-Konversion	Anteil Paarungen mit <code>contactStatus = PILOT_AGREED</code>	<code>mart_matrix</code>
Credit-Burn	Summe <code>SPEND</code> je Monat, getrennt nach <code>FIX</code> / <code>FLEX</code>	<code>mart_credits</code>
Credit-Auslastung	verbrauchte / gewährte Credits je Startup	<code>mart_credits</code>
Marktplatz-Nutzung	Buchungen je Angebotstyp; Confirm-Quote; Storno-Quote	<code>mart_marketplace</code>
Bewertungsstreuung	Standardabweichung <code>overallScore</code> je Startup über Evaluatoren	<code>mart_evaluation</code>
Sicherheitslage	fehlgeschlagene Logins/Tag, <code>PERMISSION_DENIED</code> /Tag, Exporte/Woche	<code>mart_security</code>

dbt-Tests je Modell: `unique`, `not_null` auf Schlüsseln, `relationships` auf Fremdschlüsseln, `accepted_values` auf Enums, plus fachliche Assertions (z. B. „`fct_credit_tx`-Summe je Konto entspricht `dim`-Saldo“ — dieselbe Invariante wie in §8.3).

16. Logging, Monitoring & Incident Response

16.1 Log-Ebenen

Log	Inhalt	Aufbewahrung	Ort
Audit-Log (§11.5)	Fachliche/sicherheitsrelevante Ereignisse	12 Monate DB + 12 Monate Archiv	Postgres, append-only
Applikationslog	strukturiert (JSON), <code>requestId</code> , <code>userId</code> , Route, Status, Dauer	30 Tage heiß, 12 Monate Archiv	Loki
WAF-Log	blockierte Anfragen, Rate-Limit-Treffer	90 Tage	WAF + Export
DB-Log	Verbindungen, DDL, langsame Queries, <code>pgaudit</code>	90 Tage	DB-Anbieter
System-/Container-Log	sshd, Kernel, Docker	90 Tage	Loki
Deploy-Log	wer, was, wann, Commit-SHA	12 Monate	CI

Redaction ist Pflicht: `password`, `passwordHash`, `authorization`, `cookie`, `set-cookie`, `token`, `DATABASE_URL`, `Message.body`, `note`-Felder. Ein Log, das K3-Daten enthält, ist ein zweites, schlechter geschütztes Datenlager.

16.2 Metriken & Health

- `/api/health` ist vorhanden (`SELECT 1`) und wird als Liveness/Readiness-Probe genutzt. **Erweitern** um: DB-Latenz, Migrationsstand, Version/Commit — aber **keine** internen Details für unauthentifizierte Aufrufer.
- Technisch: Fehlerquote (5xx), p95-Latenz je Route, Event-Loop-Lag, RSS, DB-Verbindungen, Replikations-Lag, Disk, Zertifikatsrestlaufzeit.
- Fachlich: Logins/h, Registrierungen/Tag, Buchungen/Tag, verarbeitete Erinnerungen, ETL-Lauf.

16.3 Alarme (mit Schwellwerten)

Alarm	Schwelle	Kanal
5xx-Rate	> 2 % über 5 min	Slack + PagerDuty-Äquivalent
<code>/api/health</code> rot	2 aufeinanderfolgende Fehlschläge	sofort
Fehlgeschlagene Logins	> 50/h global oder > 10 für ein Konto	Security-Kanal
<code>PERMISSION_DENIED</code>	> 20/h für einen Nutzer	Security-Kanal
Exporte	> 3 in 10 min durch einen Nutzer	Security-Kanal (§11.6)
Rollenänderung auf <code>ADMIN</code>	jede	sofort, immer
Recovery-Code verwendet	jede	sofort
Break-Glass-Konto genutzt	jede	sofort
DB-Verbindungen	> 80 % <code>max_connections</code>	Ops
Replikations-Lag	> 5 min	Ops
Backup fehlgeschlagen / ETL ausgeblieben	1 Vorkommnis / 26 h	Ops

Alarm	Schwelle	Kanal
Zertifikat	< 21 Tage Restlaufzeit	Ops
Neue kritische CVE in Abhängigkeiten	jede	Dev

16.4 Detection-Regeln

Konkrete Muster, die auf das Bedrohungsmodell (§4) zugeschnitten sind:

Muster	Interpretation
Login-Erfolg aus neuem Land + sofortiger Matrix-Zugriff + Export	Kontoübernahme, laufende Exfiltration
Ein Partner-User erzeugt viele <code>PERMISSION_DENIED</code> auf fremde <code>partnerId</code>	A2 testet IDOR
<code>SENSITIVE_VIEWED</code> auf > 100 verschiedene Startups in < 10 min	Scraping/Massenabzug
Rollenänderung außerhalb der Arbeitszeit	Privilegien-Missbrauch
Zugriffe eines Nutzers nach Offboarding-Datum	Offboarding unvollständig (§10.5)
Viele <code>LOGIN_FAILED</code> über viele Konten von einer <code>ipHash</code>	Credential Stuffing (A1)
DB-Verbindung aus unerwarteter Quell-IP	Netzsegmentierung verletzt

16.5 Incident Response

Rollen: Incident Lead (koordiniert, entscheidet) · Technical Lead (analysiert, behebt) · Communications Lead (Partner, Betroffene, Behörde) · Scribe (Zeitleiste). In einem kleinen Team können Rollen zusammenfallen — **aber der Incident Lead ist nie gleichzeitig Technical Lead.**

Schweregrade

Grad	Definition	Reaktion	Eskalation
SEV1	K3-Daten abgeflossen, Kontoübernahme Admin, Ransomware	sofort, 24/7	GF + DSB + ggf. Aufsichtsbehörde
SEV2	Ausnutzbare Lücke, Teilausfall, verdächtiger Zugriff	< 4 h	Tech Lead + GF
SEV3	Fehlkonfiguration ohne Abfluss	< 1 Arbeitstag	Tech Lead
SEV4	Beobachtung, Härtingsbedarf	Backlog	—

Ablauf

- Erkennen & deklarieren** — Grad festlegen, Incident-Kanal eröffnen, Zeitleiste starten.
- Eindämmen** — Konto sperren (`isActive = false`), **alle Sessions widerrufen** (`sessionsValidFrom = now()`, §9.5), betroffene Secrets rotieren, WAF-Regel setzen, im Extremfall Wartungsmodus.
- Beweise sichern — vor dem Aufräumen.** Snapshot der DB und der Logs, Audit-Events exportieren, Container-Images einfrieren. Häufigster Fehler: Neustart/Redeploy zerstört die Beweislage.
- Umfang bestimmen** — Audit-Log auswerten: Welche Entitäten wurden gelesen? Welche Exporte? Welche Partner sind betroffen? **Genau hier zählt sich §11.5 aus** — ohne Audit-Log ist diese Frage nicht beantwortbar, und man muss im Zweifel vom Worst Case ausgehen (auch gegenüber der Aufsichtsbehörde).
- Beheben & wiederherstellen** — Fix, Restore falls nötig (§8), verifizieren.
- Melden** — DSGVO Art. 33: **72 Stunden** an die Aufsichtsbehörde bei Risiko für Betroffene; Art. 34: Benachrichtigung der Betroffenen bei hohem Risiko. Betroffene Partner werden informiert, auch wenn nur Geschäftsgeheimnisse (keine PII) betroffen sind — vertraglich und für das Vertrauensverhältnis.

7. **Post-Mortem** innerhalb von 5 Arbeitstagen: blameless, mit Zeitleiste, Ursachenanalyse (5 Whys) und **terminierten** Maßnahmen mit Verantwortlichen.

Vorbereitete Runbooks (§21.4): Kontoübernahme · Datenabfluss K3 · Ransomware · Provider-Totalausfall · Secret-Leak im Repo · kompromittierte Abhängigkeit.

17. DSGVO & Compliance

17.1 Rollen

Lovedis ist **Verantwortlicher** (Art. 4 Nr. 7) für: Nutzerkonten, Kontaktdaten, Nachrichten, Bewertungen, Buchungen. Alle Infrastruktur- und Dienstleister sind **Auftragsverarbeiter** (Art. 28) und benötigen einen **AVV**.

17.2 Verzeichnis von Verarbeitungstätigkeiten (Auszug)

Verarbeitung	Zweck	Rechtsgrundlage	Kategorien	Löschfrist
Nutzerkonten & Login	Zugang, Sicherheit	Art. 6 (1) b (Vertrag)	Name, E-Mail, Firma, Rolle, Hash	Konto + 30 Tage
Login-Protokolle	Angriffserkennung	Art. 6 (1) f (berechtigtes Interesse)	IP-Hash, Zeit, Ergebnis	90 Tage
Audit-Log	Nachweispflicht, Sicherheit	Art. 6 (1) c + f	Actor, Aktion, Entität	12 Monate
Startup-Kontakte	Scouting-Kommunikation	Art. 6 (1) f	Name, Position, E-Mail, Telefon	bis Widerspruch
Bewertungen/Screening	Kerngeschäft	Art. 6 (1) b/f	Scores, Notizen	5 Jahre
Nachrichten	Zusammenarbeit	Art. 6 (1) b	Inhalt, Metadaten	12 Monate
Credit-Ledger	Abrechnung	Art. 6 (1) b + c	Buchungen	10 Jahre (§ 147 AO)
Erinnerungs-E-Mails	Prozessunterstützung	Art. 6 (1) f	Name, E-Mail	mit Erinnerung
DWH-Auswertungen	Steuerung	Art. 6 (1) f	pseudonymisiert	siehe §15.6

Zur IP-Adresse: Sie ist personenbezogen. Im Audit- und Login-Log wird deshalb nur ein **HMAC** gespeichert (`ipHash`) — das erhält die Fähigkeit, gleiche Herkunft zu erkennen (Credential Stuffing), ohne die IP im Klartext zu speichern. Der HMAC-Schlüssel wird alle 90 Tage rotiert, wodurch die Korrelierbarkeit automatisch endet.

17.3 Technische & organisatorische Maßnahmen (Art. 32)

Diese Tabelle ist die Antwort auf Partner-Fragebögen und gehört in jeden AVV-Anhang.

Kategorie	Maßnahme	Abschnitt
Pseudonymisierung	DWH-Pseudonymisierung, IP-Hashing	§15.6, §17.2
Verschlüsselung	TLS 1.3, Encryption at Rest, Feldverschlüsselung, verschlüsselte Backups	§13.4, §7.5, §8.1
Zutrittskontrolle	ISO-27001-zertifizierte Rechenzentren in DE	§5.3
Zugangskontrolle	Argon2id, MFA-Pflicht Team, Rate-Limit, Lockout, Passwort-Manager, Hardware-Keys	§9, §10.5
Zugriffskontrolle	RBAC, RLS, Least Privilege DB-Rollen, IDOR-Muster	§7.2, §7.4, §11
Weitergabekontrolle	Netzsegmentierung, Egress-Allowlist, Exportkontrolle, keine K3 an Fremdsysteme	§13.2, §11.6, §14.4
Eingabekontrolle	Append-only Audit-Log	§11.5
Verfügbarkeit	HA-DB, 2 App-Nodes, 3-2-1-Backup, geprobtetes Restore	§8

Kategorie	Maßnahme	Abschnitt
Trennungskontrolle	getrennte Umgebungen/Secrets/DBs, Mandantentrennung	§6.1, §11.4
Belastbarkeit	Rate-Limits, Timeouts, Monitoring, Alarme	§13.3, §16
Wiederherstellbarkeit	RPO/RTO definiert, monatliche Restore-Tests	§8.2, §8.3
Überprüfung	jährlicher Pentest, Quartals-Review, Post-Mortems	§19

17.4 Betroffenenrechte (Art. 15–21)

Diese müssen **implementiert**, nicht nur zugesagt werden — sonst wird jede Anfrage zu manueller DB-Arbeit mit Fehlerrisiko.

Recht	Umsetzung	Frist
Auskunft (Art. 15)	Server Action „Meine Daten exportieren“ → JSON/PDF über alle Modelle mit Bezug zum Nutzer	1 Monat
Berichtigung (Art. 16)	Selbstbedienung im Profil	unverzüglich
Löschung (Art. 17)	Zweistufig, siehe unten	1 Monat
Einschränkung (Art. 18)	<code>isActive = false</code> + Sperrkennzeichen	unverzüglich
Datenübertragbarkeit (Art. 20)	maschinenlesbarer JSON-Export	1 Monat
Widerspruch (Art. 21)	Kontaktdaten aus Scouting entfernen	unverzüglich

Löschkonzept — das ist bei diesem Schema nicht trivial:

- Stufe 1 — Anonymisierung** (sofort): `email` → `deleted-<hash>@invalid`, `name` → „Gelöschter Nutzer“, `passwordHash` → Zufallswert, `isActive = false`, `sessionsValidFrom = now()`, MFA-Secret gelöscht.
- Stufe 2 — Kaskaden prüfen:** Das Schema nutzt bewusst `onDelete: SetNull` bei `screenedById`, `handledById`, `updatedById`, `createdById` (Credit-Tx) — fachliche Datensätze bleiben also erhalten, wenn ein Nutzer verschwindet. Das ist richtig. Aber: `onDelete: Cascade` bei `Evaluation.evaluatorId`, `Message.senderId`, `PartnerStartupReview.partnerId`, `Engagement.createdById` würde bei echter Zeilenlöschung **fachliche Daten mitreißen** (Bewertungen, Verdikte, Nachrichten). → **Deshalb wird ein User nie physisch gelöscht, sondern anonymisiert.** Das erfüllt Art. 17, erhält die Geschäftshistorie und vermeidet Kaskadenschäden.
- Stufe 3 — Aufbewahrungspflichten:** `CreditTransaction` bleibt 10 Jahre (§ 147 AO, Art. 17 Abs. 3 lit. b) — aber ohne Personenbezug (`createdById` auf `null`).
- Stufe 4 — DWH:** Löschung/Anonymisierung wird in der nächsten ETL-Runde nachgezogen; `dim_user` wird über `user_key` aktualisiert. Das Löschbegehren muss also **auch das DWH** erreichen — ein häufig vergessener Schritt.
- Stufe 5 — Backups:** Backups werden **nicht** rückwirkend bearbeitet (technisch unmöglich und integritätsschädlich). Stattdessen: dokumentierte Frist (max. 12 Monate), nach der die Daten durch Rotation entfallen; bei einem Restore wird die Löschliste erneut angewendet. Diese Vorgehensweise ist aufsichtsrechtlich anerkannt, muss aber **dokumentiert** sein.

Automatisierte Löschjobs (nächtlich): Nachrichten > 12 Monate, `LoginAttempt` > 90 Tage, `PasswordResetToken` abgelaufen, `AuditEvent` > 12 Monate (→ Archiv), `IntroRequest` > 2 Jahre, `raw` / `staging` im DWH gemäß Retention.

17.5 Drittlandtransfer & Transfer-Impact-Assessment

In Szenario A ist das Thema strukturell entschärft — alle Verarbeiter sitzen in der EU. Genau das ist der Compliance-Wert der Empfehlung.

Für jeden verbleibenden Nicht-EU-Dienst ist Folgendes zu dokumentieren:

Dienst	Sitz	Grundlage	Maßnahmen
Cloudflare (falls als WAF/DNS behalten)	US	SCC + EU-US Data Privacy Framework	Data Localization Suite, TLS, TIA dokumentieren
Neon (nur in Szenario B)	US	SCC + DPF	EU-Region, Verschlüsselung, TIA
GitHub (Code, CI)	US	SCC + DPF	keine Produktionsdaten im Repo, Secret Scanning
HIBP	UK	Angemessenheitsbeschluss	k-Anonymity → kein Personenbezug übertragen
Notion / Slack (falls genutzt)	US	SCC + DPF	keine K2/K3-Daten hinterlegen

Das TIA prüft je Dienst: Art der Daten, Wahrscheinlichkeit behördlicher Zugriffe, Wirksamkeit der technischen Maßnahmen (Verschlüsselung mit Schlüsselhoheit beim Verantwortlichen), Alternativen. **Ergebnis dokumentieren und jährlich prüfen.**

17.6 Weitere Pflichten

- **AV-Verträge** mit: Hosters, DB-Anbieter, WAF, Mail-Provider, Monitoring, Passwort-Manager, Objektspeicher. Vor Produktivstart abgeschlossen.
- **Datenschutzerklärung** für Homepage und Plattform getrennt; Plattform-Erklärung muss Bewertungsdaten und die Rolle der Partner benennen.
- **Cookies:** Der Session-Cookie ist **technisch notwendig** → kein Consent nötig. Sobald Analytics/Marketing-Cookies auf lovedis.de hinzukommen, ist ein TTDSG-konformes Consent-Management erforderlich. Empfehlung: serverseitige, cookiefreie Statistik (z. B. self-hosted Plausible/Matomo in DE) — dann bleibt die Consent-Frage klein.
- **Datenschutz-Folgenabschätzung (Art. 35):** Wegen systematischer Bewertung von Startups („Scoring“), umfangreicher Verknüpfung und Profilbildung über Partner **empfohlen** — auch wenn keine besonderen Kategorien nach Art. 9 verarbeitet werden. Der Aufwand ist gering, der Nutzen im Partner-Audit hoch.
- **Datenschutzbeauftragter:** Bei < 20 Personen in der Regel nicht verpflichtend (§ 38 BDSG), wegen der Bewertungsprozesse aber prüfen; alternativ externer DSB.
- **Geschäftsgeheimnisschutz (GeschGehG):** Die Match-Matrix und Partner-Verdikte sind Geschäftsgeheimnisse **Dritter**. Zum Schutz gehören „angemessene Geheimhaltungsmaßnahmen“ — genau die Maßnahmen dieses Konzepts. Zusätzlich: NDAs mit allen Mitarbeitenden und Vertraulichkeitsklauseln in den Partnerverträgen.

18. CI/CD & Supply-Chain-Sicherheit

18.1 Pipeline (Soll)

PR eröffnet

├

Lint (ESLint) + TypeScript strict

├

Unit-/Integrationstests (Vitest) – inkl. NEGATIV-Autorisierungstests (§11.3)

├

prisma validate + Migrations-Dry-Run

├

npm audit --audit-level=high (Build bricht bei kritisch)

├

SAST (CodeQL / Semgrep: Regeln für dangerouslySetInnerHTML, \$queryRawUnsafe, fehlende require*-Guards in actions/**)

├

Secret Scanning (Gitleaks + GitHub Push Protection)

├

SBOM erzeugen (CycloneDX) + als Artefakt archivieren

├

Container-Image bauen (multi-stage, non-root, distroless/slim)

├

Image-Scan (Trivy: OS + Bibliotheken)

└

Review-Pflicht: 1 Approval, bei Auth/DB/Actions-Änderungen 2

Merge auf main

├

Deploy → staging (automatisch)

├

prisma migrate deploy (Rolle lovedis_migrate)

├

Smoke-Tests (/api/health, Login, Matrix, Booking-Flow)

└

Deploy → production (manuelles Approval, Snapshot vor Migration)

18.2 Repository-Härtung

Maßnahme	Vorgabe
Branch Protection auf <code>main</code>	keine direkten Pushes, keine Force-Pushes, Status-Checks Pflicht
Signierte Commits	<code>commit.gpgsign</code> bzw. SSH-Signaturen, „Require signed commits“
CODEOWNERS	<code>src/auth*</code> , <code>src/middleware.ts</code> , <code>src/lib/auth-guards.ts</code> , <code>prisma/schema.prisma</code> , <code>next.config.ts</code> → Review durch Security-Verantwortlichen
Actions-Pinning	Third-Party-Actions auf Commit-SHA , nicht auf Tag (Tags sind verschiebbar)
CI-Berechtigungen	<code>permissions: read-all</code> als Default, Schreibrechte nur wo nötig
Deploy-Credentials	OIDC statt langlebiger Tokens
Secret Scanning	Push Protection aktiv; bei Treffer: Secret rotieren , nicht nur Commit entfernen (History bleibt geklont)
<code>.gitignore</code>	<code>.env*</code> , <code>src/generated/</code> , <code>.open-next/</code> , Dumps, Exporte
Dependabot/Renovate	wöchentlich, Security-Updates automatisch gemerged bei grünen Tests

18.3 Abhängigkeiten mit Handlungsbedarf

Paket	Problem	Maßnahme
<code>next-auth@5.0.0-beta.31</code> (F-13)	Beta im Authentifizierungspfad; Breaking Changes zwischen Betas möglich	Version exakt pinnen (kein <code>^</code>), Changelog aktiv verfolgen, Auth-Flows durch Tests abdecken, auf stabile v5 aktualisieren sobald verfügbar

Paket	Problem	Maßnahme
<code>xlsx@0.18.5</code> (F-07)	CVE-2023-30533 (Prototype Pollution), CVE-2024-22363 (ReDoS). Die npm-Distribution wird nicht mehr gepflegt	Ersetzen durch <code>exceljs</code> , oder SheetJS aus der offiziellen Distribution beziehen. Bis dahin: Import fremder Dateien deaktivieren bzw. in Worker isolieren, Größe begrenzen
<code>html2canvas@1.4.1</code>	Seit Jahren ohne Release; verarbeitet DOM clientseitig	Für serverseitige PDF-Erzeugung (§11.6) ohnehin ersetzen
<code>bcryptjs</code>	siehe F-09	Nach Argon2id-Migration nur noch für Legacy-Verifikation vorhalten
<code>embedded-postgres</code> (beta)	nur DevDependency	akzeptabel, nie in Produktion
<code>@prisma/adapters-neon</code> , <code>@neondatabase/serverless</code> , <code>@opennextjs/cloudflare</code>	nur für Szenario B nötig	Bei Migration nach A entfernen — reduziert Angriffsfläche und Bundle

18.4 Umgang mit Datei-Importen (Kernrisiko von `xlsx`)

Der Excel-Import verarbeitet **fremde Dateien** — das ist die klassische Einfallsstelle für Parser-Schwachstellen. Unabhängig von der Bibliothek:

1. Dateigröße hart begrenzen (10 MB), Zeilen-/Spaltenzahl begrenzen.
2. Parsing in einem **separaten Prozess/Worker** mit Speicher- und Zeitlimit (kein Hauptprozess-Crash, kein ReDoS-Stillstand).
3. Ergebnis **immer** durch Zod validieren — Parser-Output nie direkt in Prisma schreiben.
4. `Object.create(null)` bzw. `Map` für dynamische Schlüssel (Prototype-Pollution-Härtung).
5. Import nur für `ADMIN` / `MEMBER`, mit Audit-Event und Vorschau vor dem Commit.

19. Umsetzungs-Roadmap

Phase 0 — Sofortmaßnahmen vor Produktivstart (P0, 1–2 Wochen)

Ohne diese Punkte sollte die Plattform keine echten Partnerdaten enthalten.

#	Maßnahme	Finding	Aufwand
0.1	Seed-Riegel für Produktion; alle Demo-Accounts entfernen; individuelle Admin-Konten anlegen	F-01	0,5 d
0.2	Rate-Limiting + Lockout am Login (Edge + App + Konto)	F-02	1,5 d
0.3	Security-Header + CSP (zunächst Report-Only)	F-05	1 d
0.4	Audit-Log-Modell + Events für Login, Matrix-Zugriff, Exporte, Rollenwechsel	F-06	2,5 d
0.5	<code>xlsx</code> ersetzen oder Import bis zur Ersetzung deaktivieren	F-07	1 d
0.6	Passwort-Policy + HIBP-Check in beiden Signup-Flows	F-14	0,5 d
0.7	Session-Lebensdauer auf 8 h + <code>__Host-</code> -Cookie	F-19	0,25 d
0.8	Passwort-Manager + MFA für alle Provider-Konten; Offboarding-Checkliste	§10.5	1 d
0.9	Backup-Verifikation: erster vollständiger Restore-Test dokumentieren	F-15	0,5 d
0.10	<code>next-auth</code> exakt pinnen; <code>npm audit</code> in CI blockierend	F-13	0,25 d

Summe Phase 0: ~9 Personentage.

Phase 1 — Hosting-Migration & Kern-Krypto (P1, 3–4 Wochen)

#	Maßnahme	Aufwand
1.1	Entscheidung Szenario A/B/C herbeiführen und dokumentieren (§20)	0,5 d
1.2	IaC + Zielumgebung aufbauen (§5.7 Schritte 1–3)	3 d
1.3	Argon2id-Migration mit Lazy-Rehash	0,5 d
1.4	MFA (TOTP) mit Pflicht für ADMIN/MEMBER	2,5 d
1.5	Passwort-Reset-Flow + EU-Mail-Provider anbinden	2 d
1.6	Session-Widerruf (<code>sessionsValidFrom</code>)	0,5 d
1.7	DB-Rollentrennung + <code>prisma migrate</code> statt <code>db push</code>	1,5 d
1.8	Secret-Store (Infisical) + Rotation aller Secrets	1,5 d
1.9	WAF vorschalten + Origin-Allowlist + Rate-Limit-Regeln	1,5 d
1.10	Monitoring/Logging/Alarmer (§16.3)	2 d
1.11	Cutover + Rollback-Fenster	1 d

Summe Phase 1: ~16 Personentage.

Phase 2 — Defense in Depth (P1/P2, 4–6 Wochen)

#	Maßnahme	Aufwand
2.1	RLS für <code>PartnerStartupMatch</code> , <code>PartnerStartupReview</code> , <code>Engagement</code> , <code>StartupPush</code>	3 d
2.2	<code>User ↔ PartnerCompany</code> -Relation (echte Mandantengrenze)	1 d
2.3	Feldverschlüsselung <code>Message.body</code> , <code>Contact.*</code> , <code>review.note</code> + Blind Index	3 d
2.4	Dateien: privater Bucket, Presigned URLs, Magic-Byte-Prüfung, Virensan	3 d
2.5	Exportkontrolle: serverseitig, Audit, Limits, Wasserzeichen	2,5 d
2.6	DB-Constraints + Append-only-Trigger	1 d
2.7	Negativ-Autorisierungstests für alle Actions + Guard-Lint-Regel	3 d
2.8	Markdown-Sanitizing für <code>ContentPage.body</code>	0,5 d
2.9	CSP von Report-Only auf blockierend	0,5 d
2.10	Automatisierte Löschjobs + Betroffenenrechte-Export	2,5 d
2.11	3-2-1-Backups inkl. Off-Site + Object Lock; monatlicher Restore-Test etabliert	1,5 d
2.12	Runbooks + IR-Plan + erste Tabletop-Übung	1,5 d

Summe Phase 2: ~23 Personentage.

Phase 3 — Data Warehouse (P2, 4–6 Wochen)

#	Maßnahme	Aufwand
3.1	<code>StartupStageEvent</code> einführen + Schreiben in bestehende Stage-Wechsel einbauen	1,5 d
3.2	DWH-Instanz, Netz, Rollen, <code>lovedis_readonly</code>	1 d
3.3	Replikation/Ingestion → <code>raw</code> inkl. Lauf-Monitoring	2,5 d
3.4	dbt-Setup, <code>staging</code> -Layer inkl. Pseudonymisierung	3 d
3.5	<code>core</code> : Dimensionen (SCD2 für <code>dim_startup</code>) + Fakten	5 d
3.6	<code>mart</code> : Funnel, Partner, Matrix, Credits, Marktplatz, Security	4 d
3.7	dbt-Tests + fachliche Assertions	1,5 d
3.8	Metabase, SSO/WireGuard-Zugang, Dashboards	2,5 d
3.9	Staging-Maskierungsskript	1 d
3.10	DWH-Governance dokumentieren (Kennzahl-Definitionen, Retention)	1 d

Summe Phase 3: ~23 Personentage.

Phase 4 — Reifegrad & Nachweis (laufend)

Maßnahme	Frequenz
Externer Penetrationstest (Fokus: Rollenmodell, IDOR, Mandantentrennung)	jährlich, erstmals nach Phase 2
Abhängigkeits- und CVE-Review	monatlich
Zugriffsrechte-Review (wer hat <code>ADMIN</code> ? wer hat DB-Zugang?)	quartalsweise

Maßnahme	Frequenz
Restore-Test	monatlich
DR-Übung	jährlich
Security-Awareness (Phishing-Simulation)	halbjährlich
Konzept-Review dieses Dokuments	halbjährlich
Optional: ISO 27001 / TISAX-Vorbereitung, falls Partner es fordern	nach Bedarf

Aufwandsübersicht

Phase	Personentage	Kalenderzeit
Phase 0 (P0)	~9	1–2 Wochen
Phase 1	~16	3–4 Wochen
Phase 2	~23	4–6 Wochen
Phase 3 (DWH)	~23	4–6 Wochen
Gesamt	~71 PT	3–4,5 Monate

Phase 3 kann parallel zu Phase 2 laufen, wenn zwei Personen verfügbar sind.

20. Offene Entscheidungen

#	Entscheidung	Optionen	Empfehlung	Wer	Blockiert
1	Hosting-Szenario	A (Hetzner/EU) · B (Cloudflare/Neon) · C (AWS FRA) · F (Scaleway)	A	GF + Tech Lead	Phase 1 komplett
2	DB-Anbieter	Aiven FRA (EU-Konzern) · IONOS/STACKIT (100 % DE) · selbst betrieben	Aiven , bei DE-Vertragspflicht IONOS	Tech Lead	1.2
3	WAF	Myra (DE, BSI-C5) · Link11 (DE) · Cloudflare (US, günstig)	Myra , wenn Budget; sonst Cloudflare mit TIA	GF	1.9
4	MFA-Verfahren	TOTP · WebAuthn/Passkeys · beides	TOTP jetzt, Passkeys als P2	Tech Lead	1.4
5	Session-Strategie	JWT kurz + <code>sessionsValidFrom</code> · DB-Sessions	JWT + Widerruf (Edge-Middleware bleibt nutzbar)	Tech Lead	1.6
6	Secret-Store	Infisical self-hosted · Vault · SOPS+age	Infisical	Tech Lead	1.8
7	Feldverschlüsselung: Umfang	nur <code>Message.body</code> · zusätzlich <code>Contact.*</code> und <code>note</code> -Felder	erweitert (Contact + Notes)	DSB + Tech Lead	2.3
8	RLS-Umfang	nur die 4 kritischen Tabellen · alle Tabellen	4 kritische zuerst , dann ausrollen	Tech Lead	2.1
9	E-Mail-Provider	Brevo (FR) · Mailjet (FR) · Scaleway TEM (FR) · self-hosted	Brevo (AVV, EU, gute API)	Tech Lead	1.5
10	Team-Passwort-Manager	1Password Business · Bitwarden (EU) · Vaultwarden self-hosted	1Password (bester Recovery-/Admin-Workflow)	GF	0.8
11	DWH-Technologie	Postgres + dbt · ClickHouse · Managed (BigQuery/Snowflake)	Postgres + dbt	Tech Lead	Phase 3
12	Nachrichten-Aufbewahrung	12 Monate · 24 Monate · unbegrenzt	12 Monate	GF + DSB	2.10
13	DSFA (Art. 35)	durchführen · begründet verzichten	durchführen (Scoring/Profilbildung)	DSB	§17.6
14	Pentest-Budget/Zeitpunkt	nach Phase 2 · nach Phase 3 · keiner	nach Phase 2	GF	Phase 4
15	<code>/odie</code> -Route in Produktion	behalten · entfernen · hinter Login	hinter Login oder entfernen	Tech Lead	0.3

21. Anhang

21.1 Environment-Variablen (Soll, Szenario A)

```
# --- Laufzeit ---
NODE_ENV=production
NEXTAUTH_URL=https://app.lovedis.de      # kein Secret
AUTH_TRUST_HOST=true                    # kein Secret

# --- Secrets (aus dem Secret-Store injiziert, NIE im Repo) ---
DATABASE_URL=                          # Rolle lovedis_app, sslmode=verify-full
AUTH_SECRET=                          # = NEXTAUTH_SECRET
CRON_SECRET=                          # Bearer für /api/cron/reminders
FIELD_KEY_V1=                         # base64, 32 Byte – Feldverschlüsselung
BLIND_INDEX_KEY=                      # base64, 32 Byte – HMAC für Suchindizes
IP_HASH_KEY=                          # base64, 32 Byte – IP-Pseudonymisierung
EMAIL_API_KEY=                        # EU-Mail-Provider
S3_ENDPOINT=
S3_BUCKET=
S3_ACCESS_KEY=
S3_SECRET_KEY=

# --- Nur in CI ---
DATABASE_URL_MIGRATE=                  # Rolle lovedis_migrate

# --- Nur im ETL-Kontext (nicht in der App!) ---
DWH_SOURCE_URL=                       # Rolle lovedis_readonly
DWH_DATABASE_URL=
DWH_PSEUDONYM_KEY=

# --- Telemetrie ---
SENTRY_DSN=
OTEL_EXPORTER_OTLP_ENDPOINT=
```

Entfällt gegenüber heute: `CLOUDFLARE_API_TOKEN`.

21.2 Go-Live-Checkliste

Identität & Zugang

- ☐ Keine Demo-Accounts, kein Seed-Passwort in der Prod-DB
- ☐ Argon2id aktiv; Legacy-bcrypt nur für Lazy-Rehash
- ☐ MFA für alle `ADMIN` / `MEMBER` eingerichtet und **verifiziert**
- ☐ Rate-Limit + Lockout getestet (bewusst gesperrt und entsperrt)
- ☐ Passwort-Reset-Flow Ende-zu-Ende getestet
- ☐ Session-Widerruf getestet (Logout überall wirkt sofort)

Daten & DB

- ☐ DB ohne öffentliche IP; `sslmode=verify-full` im DSN
- ☐ Vier DB-Rollen aktiv; App-Rolle ohne DDL
- ☐ RLS auf den vier kritischen Tabellen scharf
- ☐ DB-Constraints + Append-only-Trigger aktiv

- ☐ Feldverschlüsselung aktiv; Schlüssel im Secret-Store
- ☐ `prisma migrate deploy` in der Pipeline; `db push` in Prod unmöglich

Backup & DR

- ☐ PITR aktiv (14 Tage); Off-Site-Dump in anderem Account
- ☐ Monatsarchiv mit Object Lock
- ☐ **Restore erfolgreich durchgeführt und protokolliert**
- ☐ Credit-Invarianten-Query nach Restore: 0 Zeilen

Anwendung

- ☐ CSP blockierend, keine Verstöße im Report
- ☐ HSTS, X-Frame-Options, Referrer-Policy, `poweredByHeader: false`
- ☐ Jede Server Action: Guard + `.strict()` -Zod + Audit
- ☐ Markdown sanitized; kein `dangerouslySetInnerHTML` ohne Begründung
- ☐ Uploads: privater Bucket, Presigned URLs, Magic Bytes, Virensan
- ☐ Exporte serverseitig, auditiert, limitiert, mit Wasserzeichen
- ☐ `xlsx` ersetzt

Netz & Edge

- ☐ WAF aktiv, Regeln blockierend, Rate-Limits gesetzt
- ☐ Origin nur von WAF-IPs erreichbar (**verifiziert durch Direktaufruf der Origin-IP**)
- ☐ Egress-Allowlist aktiv
- ☐ TLS 1.3, CAA, DNSSEC, SPF/DKIM/DMARC (`p=reject`)
- ☐ `__Host-`-Cookie; kein domainweites Session-Cookie

Betrieb

- ☐ Audit-Log schreibt alle Ereignisse aus §11.5
- ☐ Alarmer aus §16.3 aktiv und **getestet** (Testalarm ausgelöst)
- ☐ Runbooks vorhanden; IR-Rollen benannt; Erreichbarkeit geklärt
- ☐ Alle Secrets rotiert (keine Entwicklungswerte in Prod)

Compliance

- ☐ AVV mit allen Auftragsverarbeitern
- ☐ VVT und TOM-Dokumentation aktuell
- ☐ Datenschutzerklärung Plattform veröffentlicht
- ☐ Löschjobs aktiv; Betroffenenrechte-Export funktioniert
- ☐ TIA für verbleibende Nicht-EU-Dienste dokumentiert

21.3 Rollen & Verantwortlichkeiten

Rolle	Verantwortung
Geschäftsführung	Risikoakzeptanz, Budget, Entscheidungen 1/3/10/12/13/14

Rolle	Verantwortung
Tech Lead / Security Owner	Architektur, Umsetzung, Reviews, Incident Lead
Ops-Verantwortlicher	Infrastruktur, Backups, Restore-Tests, Monitoring
Datenschutzbeauftragter (intern/extern)	VVT, DSFA, Betroffenenrechte, Meldungen
Entwicklungsteam	Secure Coding, Guards, Tests, Dependency-Hygiene
Alle Mitarbeitenden	Passwort-Manager, MFA, Meldung von Verdachtsfällen

21.4 Benötigte Runbooks

Runbook	Kerninhalt
Kontoübernahme	Konto deaktivieren, <code>sessionsValidFrom</code> setzen, Audit-Log auswerten, MFA neu einrichten, Nutzer informieren
Datenabfluss K3	Beweissicherung vor Aufräumen, Umfang über Audit-Log, betroffene Partner identifizieren, 72-h-Meldefrist prüfen
Ransomware	Isolieren, aus Monatsarchiv neu aufbauen, alle Secrets rotieren, kein Restore in kompromittierte Umgebung
Secret-Leak im Repo	Secret rotieren (nicht nur Commit löschen), Zugriffs-Logs auf Missbrauch prüfen, Push Protection nachschärfen
Fehlerhafte Migration	PITR auf Zeitpunkt vor Migration, Diff der verlorenen Änderungen, Nachfahren
Provider-Ausfall	Statusseite prüfen, Kommunikation an Nutzer, ggf. Aufbau beim Alternativenanbieter
Kompromittierte Abhängigkeit	SBOM-Abgleich, betroffene Version pinnen, Image neu bauen, Ausrollen
Restore-Test (Routine)	Wegwerf-DB, Dump laden, Verifikationsqueries (§8.3), Protokoll

21.5 Bezug zu bestehenden Dokumenten

Dokument	Verhältnis zu diesem Konzept
<code>docs/deployment-plan-mara.md</code>	Beschreibt das isolierte Staging-Deployment auf Cloudflare. Bleibt für Staging gültig. Der dort offene <code>CLOUDFLARE_API_TOKEN</code> -Blocker (§6) entfällt bei Wahl von Szenario A. Der Grundsatz „Live-Homepage nicht anfassen“ wird hier übernommen und um Cookie-Isolation erweitert (§14.3).
<code>docs/mara-implementation-notes.md</code>	Dokumentiert die offenen Infrastruktur-Stellen: E-Mail-Adapter (<code>consoleEmailAdapter</code>) und fehlenden Cron-Trigger. Beide werden hier verbindlich aufgelöst (§14.4).
<code>README.md</code>	Enthält die Demo-Accounts und das Seed-Passwort. Muss um einen Warnhinweis ergänzt werden , dass diese ausschließlich für lokale Entwicklung gelten (F-01).
<code>prisma/schema.prisma</code>	Wird um <code>AuditEvent</code> , <code>LoginAttempt</code> , <code>PasswordResetToken</code> , <code>StartupStageEvent</code> , MFA- und Session-Felder sowie die <code>User ↔ PartnerCompany</code> -Relation erweitert.
<code>docs/plan-match-matrix.md</code>	Die dort eingeführte Matrix ist der K3-kritischste Datensatz; dieses Konzept ergänzt Audit, RLS und Exportkontrolle dafür.

Dokumentende. Änderungen an diesem Konzept erfordern ein Review durch den Security Owner. Nächste planmäßige Überprüfung: **Januar 2027**.